

QA Test Guide — Step by Step

Printable walkthrough generated from [docs/QA_TEST_PLAN.md](#). Each case shows its Gherkin scenario and the plain-English steps. Record results on the companion [QA Run Log](#) sheet. Generated 2026-07-09.

End-to-end manual test plan covering the **entire** functionality of the app across all three clients: **Web** (Blazor WASM), **Desktop** (MAUI / Windows), and **Android** (MAUI). Each case is written twice — a **Gherkin** scenario (Given/When/Then, matching this project's convention so it can later seed the automated E2E .Tests Playwright suite) and a ****plain-English walkthrough**** a manual tester can follow step by step.

Brand in examples is "Perezosoft"; substitute your app's brand if rebranded.

How to use this document

- **Run the Smoke suite (§4) first.** It's the ~15-minute critical path. If any smoke case fails, stop and report — deeper suites will likely cascade.
- **Each case has an ID** (e.g. QA-AUTH-03). Record the result against the ID using the **sign-off sheet (§14)**: Pass / Fail / Blocked / N-A, plus tester, build/commit, date, notes.
- **Priority:** • **Smoke** Smoke (critical path) • • **Core** Core (run every regression) • • **Edge** Edge (run on full regression or when the area changed).
- **Automated in CI** on a case title means a Playwright journey in `tests/E2E.Tests` now exercises the same path on every push (the e2e job in `.github/workflows/ci.yml`). Human QA can **spot-check** these rather than run them in full each cycle; they still need a manual pass on Desktop/Android (the E2E job runs Web only) and whenever the area changes. See §15 for the exact list and §17 for the run procedure.
- **Both formats describe the same test.** Read whichever suits you; the Gherkin is the source of truth for automation.
- **"App" = whichever client the suite header names.** Most behavior is identical across clients (same API, same shared RCL UI); the per-client suites (§11–12) only cover what genuinely differs — the auth transport and session persistence.

1. Environment & prerequisites

1.1 Backing services + API (host machine)

```
docker compose up -d # Postgres + Mailpit
dotnet run --project src/Api --launch-profile https # binds https:7160 (web/desktop) AND http:5238 (android)
```

- API health/liveness check: `curl -k https://localhost:7160/health -> 200` (Healthy); `curl -k https://localhost:7160/health/ready -> 200` when the database is reachable (503 if not). *(The older `curl -k -X POST https://localhost:7160/api/auth/refresh -> 401` reachability check still works.)*
- **Mailpit UI:** <http://localhost:8025> — this is the dev mail trap. Every magic link, OTP code, and invitation email lands here. Keep it open in a tab throughout testing.

(!) **Email only reaches Mailpit if SMTP points at it.** If your repo-root `.env` has the `Email__Smtp__*` lines set to a real provider (e.g. Brevo), the API sends auth emails there and **Mailpit stays empty** — every email-based case below will appear to "fail." For QA, route mail to Mailpit by **either**:

- commenting out the `Email__Smtp__*` lines in `.env` (unset -> defaults to Mailpit `localhost:1025`), **or**
- leaving `.env` untouched and overriding on the command line (command-line config beats `.env`):

```
```bash
dotnet run --project src/Api --launch-profile https -- \
--Email:Smtp:Host=localhost --Email:Smtp:Port=1025 --Email:Smtp:Username= --Email:Smtp:Password=
```
```

Verify by triggering one OTP (QA-SMK-01) and confirming it appears in Mailpit before running the suite.

(!) **Email delivery is now asynchronous.** Requesting a code/link/invite **enqueues** the email and a background dispatcher (the outbox) sends it — so it appears in Mailpit ****a few seconds later**, not instantly. **Wait briefly before assuming failure.** The **send request now** always returns success** (reliability moved to the background): if an email never arrives while SMTP points at Mailpit, the message is retrying or dead-lettered in the `OutboxMessages` table — it is **no longer** surfaced as a request error.

- Web app: `dotnet run --project src/Web --launch-profile https -> https://localhost:7008.`

(!) Always use the **https** profile for both web and API. Chrome treats `http://localhost` and `https://localhost` as different sites, so the refresh cookie is dropped over http and sign-in silently fails to persist. (See `docs/DECISIONS.md` / the `schemeful-same-site` note.)

(!) **The passwordless endpoints are rate-limited** (default **5 requests/minute per IP** on `/otp/send`, `/magic-link/send`, and `/otp/verify`). If you fire many code/link requests or verify attempts in quick succession you may get **HTTP 429** — that's the abuse guard working (QA-AUTH-11), **not** a bug. Pace requests, or wait ~1 minute for the window to reset.

1.2 Test accounts & data

| Need | What to prepare |
|--------------------------------|--|
| Two real Google accounts | e.g. <code>qa.owner@gmail.com</code> , <code>qa.member@gmail.com</code> — for OAuth + linking + multi-user household flows. |
| One Microsoft personal account | for the Microsoft OAuth path (provider pinned to the <code>*consumers*</code> tenant). |
| Throwaway email addresses | any address works for magic-link / OTP — mail is trapped by Mailpit, so the address need not be real. Use distinct ones per test to keep inboxes clean. |
| Two browser contexts | a normal window and an incognito/second-profile window. The household invite flow needs two <code>*different*</code> signed-in users at once; incognito gives you an isolated session + cookie jar. |

Database reset between full runs (optional but recommended): to retest "new user" onboarding cleanly, you need users that don't yet exist. Either use fresh email addresses each run, or reset the dev DB: `docker compose down -v && docker compose up -d`, then re-apply migrations by starting the API. A volume wipe destroys all test data — only do it on the dev environment.

1.3 Desktop (MAUI Windows) — additional setup

- Run the app from Visual Studio (Windows Machine target) or `dotnet build src/Maui -t:Run -f net10.0-windows...`
- API must be running on `https://localhost:7160` (the desktop client's base URL).
- OAuth uses a **loopback browser flow** — your default system browser will open a tab during OAuth.

1.4 Android (MAUI) — additional setup

Follow `docs/MOBILE_TESTING.md`. The essential bits:

- Emulator (AVD) or USB device running.
- ****adb reverse tcp:5238 tcp:5238 — re-run every time the device/emulator restarts**** (it does not persist). Verify with `adb reverse --list`.
- API started with the **https** profile (binds the cleartext : 5238 leg the device uses).

- Provider redirect URIs registered: `http://localhost:5238/signin-google` and `http://localhost:5238/signin-microsoft`.

1.5 Environment B — deployed staging (DEPLOY, ADR-017)

Everything above is **Environment A** (local). The same suite also runs against the **deployed staging** environment — one Render container serving the API + WASM **single-origin** over real HTTPS, backed by Neon Postgres and Brevo email. Point the browser at the staging URL (e.g. `https://<app>-staging.onrender.com`) and execute the cases exactly as written. What differs from local:

- **One origin, real TLS.** Web + API share the host, so there's no separate API port and no CORS step.
- **Email is real (Brevo), not Mailpit.** Use **real or plus-addressed inboxes** you can open (e.g. `you+qa1@gmail.com`); there's no mail-trap UI. If a code/link doesn't arrive, check **Brevo -> Transactional -> Logs** (and that a verified sender + `Email__SmtP__FromAddress` are set). Email cases are therefore slower than local — pace them (the passwordless rate limit still applies).
- **Cold start.** Free instances sleep after ~15 min idle; the **first** request of a session can take ~30–60 s. That's expected, not a failure — retry once it wakes.
- **OAuth** buttons appear **only** for providers configured on staging (Google/Microsoft need their redirect URIs registered against the staging host). If unconfigured, the buttons are simply absent (by design) — record those provider cases **N-A** for this environment.
- **Billing** uses Stripe **test mode** — exercise webhooks with `stripe trigger ...` against the staging `/api/billing/webhook`.

Auto-deploy + smoke gate. A merge to develop that passes CI auto-deploys staging, waits for the new build to be live (`/api/version` reports the pushed commit), and runs an automated post-deploy smoke (liveness/readiness, SPA shell + deep-link, `/api` returns an API-shaped 404, `/api/auth/providers`). A red smoke blocks — so a broken deploy is caught before manual QA starts. Manual QA on staging complements it (the human-only paths: real email, OAuth, billing, visual checks).

2. Scope

In scope: authentication (all methods, all clients), new-user onboarding, household/tenant management, invitations & joining, account settings (linked providers), localization (EN/ES), transactional emails, cross-cutting security (tenant isolation, auth guards, token lifecycle), and **native (MAUI) feature parity** — the shared-RCL feature surface exercised per platform on Windows + Android (§12–13) with a first-run iOS/macCatalyst smoke (§13b) and a per-release native checklist (§13c).

****Out of scope (per docs/PROJECT_BRIEF.md OUT list & current state):**** SMS OTP, OAuth providers beyond Google/Microsoft, FR/DE/PT languages (scaffolded but not translated — see docs/LOCALIZATION.md), and any app-specific domain features not yet built on this platform.

Platform services with no client UI (API/operational-level, not manually testable through the app yet): the billing API (`/api/billing/*`), the append-only audit log, OpenTelemetry telemetry, the health endpoints, the background outbox/inbox/scheduled-jobs, and **file storage** — the `IFileStorage` seam + the signed download endpoint `GET /api/files/{token}` (anonymous, the token `*is*` the authorization; local-disk only — cloud backends hand out native presigned URLs). These are **covered by automated tests** (`tests/Api.Tests`); E2E is pending. Health has a smoke check (QA-SMK-07); manual cases for the rest will be added when client UI exists. **GDPR data export** (`POST /api/household/export`) and **account erasure** (`DELETE /api/auth/me`) now **have a web UI** (UI-1: owner Household -> Data, Settings -> Danger zone) — covered by the manual cases QA-HH-13 + QA-SET-07. **RBAC role management now has a web UI** (RBAC-3) — covered by the household cases QA-HH-09..12. **MFA now has a web UI** (UI-2) — enrollment/disable in Settings and the sign-in step-up on Login — covered by QA-MFA-01..03. The **in-app notification center now has a web UI** (UI-3) — the header bell (list, unread count, mark-read) and Settings delivery-preference switches — covered by QA-NOTIF-01..03. The **platform-staff admin surface now has a web UI** (UI-4) — a staff-only `/admin` console (tenant list/detail + impersonation) — covered by QA-ADMIN-01..03. The **public API (PUBAPI)** and **outbound webhooks (HOOKS)** are intentionally UI-less (they're for machines) and **config-gated off** — they have **manual curl/Postman cases in §14b** (QA-API-01..06), in addition to automated tests.

Automated in CI (Web): a Playwright/PHPUnit E2E suite (`tests/E2E.Tests`) now runs the core auth, MFA, and i18n journeys against the real bootied stack on every push — the e2e job in `.github/workflows/ci.yml`. The cases it covers are marked **Automated in CI** (QA-SMK-01, QA-SMK-03, QA-AUTH-09, QA-MFA-01, QA-MFA-02, QA-I18N-01;

see the \$15 note). Human QA can spot-check those on Web and focus effort on the un-automated cases and the Desktop/Android clients, which the CI job does not exercise.

3. Test data conventions

- **Owner user** = the account you sign in with first; it auto-creates and owns a household.
- **Member user** = a second account invited into the owner's household.
- Household auto-named on creation; rename it to something recognizable (e.g. "QA House") early so you can spot it in the header tenant badge.

4. Smoke suite • Smoke (run first — ~15 min)

The critical path: can a user get in by each method, reach the app, and get out.

QA-SMK-01 — Web: OTP sign-in happy path • Smoke (Web) Automated in CI

Gherkin

```
Given I am an anonymous user on the /login page of the web app
When I enter my email and request a 6-digit code
And I retrieve the code from Mailpit and submit it
Then I am signed in and land on the home page
And the header shows my household name and my display name
```

Walkthrough

- 1 Open <https://localhost:7008> -> you're redirected to /login.
- 2 In the email field enter `qa-smoke@example.com`; click **Email me a 6-digit code**.
- 3 **Expected:** the form switches to a code-entry view ("Enter the code we sent to...").
- 4 Open Mailpit (<http://localhost:8025>); open the newest mail; copy the 6-digit code.
- 5 Enter the code; click **Verify code**.
- 6 **Expected:** you land on the home page; the top header shows a tenant badge (household name) and your display name, plus **Household**, **Settings**, **Sign out** buttons.

QA-SMK-02 — Web: Google OAuth sign-in • Smoke (Web)

Gherkin

```
Given I am on the /login page
When I click "Continue with Google" and complete Google consent
Then I am returned to the app, signed in, on the home page
```

Walkthrough

- 1 From /login, click **Continue with Google**.
- 2 **Expected:** full-page redirect to Google's consent screen.
- 3 Choose `qa.owner@gmail.com`, approve.
- 4 **Expected:** redirected back into the app, signed in, on the home page with the header chrome.

QA-SMK-03 — Web: Sign out • Smoke (Web) Automated in CI

Gherkin

```
Given I am signed in to the web app
When I click "Sign out"
Then I am returned to the /login page
And navigating to /settings redirects me back to /login
```

Walkthrough

- 1 While signed in, click **Sign out** in the header.
- 2 **Expected:** you land on /login.
- 3 In the address bar go to /settings.

- 4 **Expected:** you're bounced back to /login (no access without a session).

QA-SMK-04 — Web: Session persists across reload ("remember me") • Smoke (Web)

Gherkin

```
Given I am signed in to the web app
When I reload the page (or reopen the tab)
Then I am still signed in without re-authenticating
```

Walkthrough

- 1 Signed in, press F5 / reload.
- 2 **Expected:** brief load, then the app shell — still signed in, no trip to /login. (A silent refresh exchanges the refresh cookie for a new access token on load.)

QA-SMK-05 — Desktop: OTP sign-in • Smoke (Desktop) — see QA-DSK-01

QA-SMK-06 — Android: OTP sign-in • Smoke (Android) — see QA-AND-01

QA-SMK-07 — API health & readiness • Smoke (Platform)

Gherkin

```
Given the API is running
When I GET /health and /health/ready
Then /health returns 200 "Healthy"
And /health/ready returns 200 when the database is reachable, 503 when it is not
```

Walkthrough

- 1 `curl -k https://localhost:7160/health -> 200`, body Healthy (liveness — process up).
- 2 `curl -k https://localhost:7160/health/ready -> 200` (readiness — Postgres reachable).
- 3 **(Optional)** stop the DB (`docker compose stop db`) and re-run step 2 -> **503**; then `docker compose start db` and confirm it returns to **200**.
- 4 **Expected:** liveness is 200 whenever the process runs; readiness tracks DB reachability. Responses are **status-only** (no connection details leaked).

5. Web — Authentication • Core

QA-AUTH-01 — Magic-link sign-in happy path • Core (Web) Automated in CI

Gherkin

```
Given I am an anonymous user on /login
When I enter my email and request a magic link
And I open the emailed link from Mailpit
Then I am signed in and landed in the app
```

Walkthrough

- 1 On /login enter `qa-magic@example.com`; click **Send me a magic link**.
- 2 **Expected:** a success panel — "Check your inbox... we sent a link to `qa-magic@example.com`" — with a **Use a different email** link.
- 3 In Mailpit open the newest mail; click the sign-in link (or copy it into the same browser).
- 4 **Expected:** the link resolves and you end up signed in, in the app.

QA-AUTH-02 — Microsoft OAuth sign-in • Core (Web)

Gherkin

```
Given I am on /login
When I click "Continue with Microsoft" and complete consent
Then I am returned signed in
```

Account resolution (read before testing): signing in with a provider whose email matches an existing account links the provider to that account rather than creating a duplicate — but only when the email is **verified**. Google asserts this; Microsoft on the ****consumers**** (personal MSA) tenant is trusted to have a verified email even though it omits the claim (IProviderEmailTrust). For **work/school** tenants (organizations/common/a tenant GUID) and any other provider, a same-email auto-link is **refused** (fail-closed takeover guard, audit MITI-3) — the user signs in with their original method and links the provider from **Settings** instead (QA-SET-02). To test the plain first-time path below, use a Microsoft account whose email has **no** prior account.

Walkthrough

- 1 Click **Continue with Microsoft**; sign in with a **personal** Microsoft account.
- 2 **Expected:** returned to the app signed in. A brand-new email creates a new account; a personal Microsoft account whose email already exists auto-links to that account (consumers tenant). (If you see a tenant/reply-URL error, the provider registration is the cause — out of app scope, note it.)

QA-AUTH-03 — OTP wrong code is rejected • Core (Web)

Gherkin

```
Given I requested an OTP code for my email
When I submit an incorrect 6-digit code
Then I see an "incorrect code" error and remain on the code-entry screen
```

Walkthrough

- 1 Request an OTP (as QA-SMK-01) but **do not** use the real code.
- 2 Enter 000000; click **Verify code**.
- 3 **Expected:** inline error ("code is incorrect / verification failed"); you stay on the code-entry view and can retry. You are **not** signed in.

QA-AUTH-04 — OTP cumulative lockout & code expiry • Edge (Web)

Gherkin

```
Given I requested an OTP code
When I submit wrong codes up to the cumulative limit (default 5 failures per email within a 15-min window)
Then the email is locked — further attempts, INCLUDING a freshly requested code, are rejected until the window elapses
And separately, an unused code is rejected once it passes its lifespan (default 10 minutes)
```

Walkthrough

- 1 Request an OTP for an email; enter a **wrong** 6-digit code until you hit the cumulative limit (default 5, Auth:Otp:MaxAttempts, counted ****per email across the Auth:Otp:LockoutWindowMinutes = 15-min window****, not per code).
- 2 **Expected:** after the limit, a "too many attempts" lockout error.
- 3 Now request a **fresh** code and submit it — even the **correct** one.
- 4 **Expected:** still rejected — requesting a new code does **NOT** reset the budget (this is the brute-force defense; a resend can't hand the attacker another N guesses). The lock clears after the 15-min window.
- 5 **Expiry sub-case:** separately, request a code and wait past Auth:Otp:CodeLifespanMinutes (default 10) — the stale code is rejected.

Note (enumeration): OTP verify returns the **same** generic "invalid code" error whether the code was wrong or there is no active code — it never reveals whether an address has an outstanding OTP. *(Long-wait cases — lower the config to test fast. Mind the per-IP rate limit, QA-AUTH-11: 5 wrong verifies in a minute also trips the 429 throttle.)*

QA-AUTH-11 — Passwordless endpoints are rate-limited • Core (Web)

Gherkin

Given I rapidly request codes/links (or verify attempts) for the same client
When I exceed the per-IP limit (default 5 per minute)
Then further requests are rejected with HTTP 429 until the window resets

Walkthrough

- 1 From /login, request an OTP (or magic link) repeatedly in quick succession — more than 5 within a minute.
- 2 **Expected:** after the limit the request is throttled (**HTTP 429**), surfaced on /login as **"Too many requests. Please wait a minute, then try again."** The same throttle — and the same message — applies to **OTP verify**.
- 3 Wait ~1 minute; requests succeed again.

Protects against email-bombing and OTP brute-forcing. Expected behavior, not a defect — see the §1.1 pacing note.

QA-AUTH-05 — Magic link is single-use • Edge (Web) Automated in CI

Gherkin

Given I signed in by clicking a magic link
When I click the same link a second time
Then it is no longer valid

Walkthrough

- 1 Complete QA-AUTH-01. Then revisit the **same** link from Mailpit.
- 2 **Expected:** it does not grant a second session — you're sent to /login with an invalid-link indication (or simply not signed in). The token is consumed on first use.

QA-AUTH-06 — Magic link expiry • Edge (Web)

Walkthrough: request a link, wait past `Auth:MagicLink:TokenLifespanMinutes` (default 15), then open it.

Expected: rejected as expired; user lands on /login. *(Lower the config to test fast.)*

QA-AUTH-07 — User-enumeration protection • Core (Web)

Gherkin

Given an email address that has no account
When I request a magic link or OTP for it
Then the UI shows the same success/"check your inbox" response as for a real account

Walkthrough

- 1 Request a magic link **and** an OTP for a never-before-used address.
- 2 **Expected:** identical success messaging to a known address — the app never reveals whether an account exists. (Mailpit will still show whatever the system chose to send.)

QA-AUTH-08 — Login error banners • Edge (Web)

Gherkin

Given the login page is loaded with an error query parameter
Then a human-readable error banner is shown

Walkthrough — visit each URL and confirm a red banner with sensible copy:

- /login?error=external_failed -> external sign-in failed message.
- /login?error=email_unverified -> email-unverified message.
- /login?error=invalid_link -> invalid/expired link message.
- /login?error=somethingelse -> generic "something went wrong" fallback.

QA-AUTH-09 — Email-format validation • Edge (Web) Automated in CI

Walkthrough: on /login, click a send button with an empty or malformed email (e.g. abc). **Expected:** inline "enter a valid email" validation; no request sent.

QA-AUTH-10 — Magic link & OTP available on web; OAuth always • Edge (Web)

Walkthrough: confirm the web login page shows **both** "Send magic link" and "Email me a code", plus Google/Microsoft buttons. (Native clients hide magic link — covered in §11–12.)

6. Web — Onboarding & new tenant • Core

QA-ONB-01 — First-ever sign-in auto-creates a household, user is owner • Smoke (Web) Gherkin

```
Given an email/account that has never signed in before
When I authenticate for the first time (any method)
Then a new household is created and I am its owner
And the header shows that household and my name
```

Walkthrough

- 1 Sign in with a brand-new account/email (fresh address or post-DB-reset).
- 2 **Expected:** you reach the app immediately (no "create household" prompt — provisioning is automatic). Open **Household:** you are listed with the **Owner** badge and are the only member.

QA-ONB-02 — Returning user keeps their household • Core (Web)

Walkthrough: sign out and back in with the same account. **Expected:** same household, same role, data intact.

7. Web — Household management • Core

Roles: **owner** > **admin** > **member** (ADR-009). **Owner + admin** can rename and invite/remove members; **owner-only:** change member roles (promote/demote), transfer ownership, dissolve. Members see a read-only name and a **Leave** button. The owner's row never shows action buttons.

QA-HH-01 — Owner renames the household • Core (Web)

Gherkin

```
Given I am the household owner on /household
When I change the name and click Rename
Then the new name is saved and reflected in the header tenant badge
```

Walkthrough

- 1 **Household** -> edit the name field (e.g. "QA House") -> **Rename**.
- 2 **Expected:** success banner; the header tenant badge updates to the new name (a silent refresh updates the claim).

QA-HH-02 — Member sees read-only household, no owner controls • Core (Web) Automated in CI

Precondition: signed in as a *member* (use QA-INV flow to create one). **Walkthrough:** open **Household**. **Expected:** household name shown as read-only heading; no rename/invite/transfer controls; a **Leave** button is present.

QA-HH-03 — Owner removes a member • Core (Web) Automated in CI

Gherkin

```
Given I am the owner and another member exists
When I click Remove next to that member and confirm
Then they are removed from the household member list
```

Walkthrough

- 1 As owner with a member present, click **Remove** by the member's row.
- 2 **Expected:** a confirm dialog ("Remove <name>?"); on confirm, success banner and the member disappears from the list. (The removed user is re-homed to a fresh solo household — verify by signing in as them: they now own an empty household — see QA-HH-08.)

QA-HH-04 — Owner cannot remove themselves • Edge (Web)

Walkthrough: as owner, confirm there is **no Remove button on your own row**. (Owner departs via transfer or dissolve, not removal.)

QA-HH-05 — Transfer ownership • Core (Web) Automated in CI

Gherkin

```
Given I am the owner and at least one other member exists
When I select that member and click Transfer
Then they become owner and I become a regular member
```

Walkthrough

- 1 As owner with ≥ 1 other member, in **Transfer ownership** pick the member -> **Transfer**.
- 2 **Expected:** success banner. The chosen member now shows the **Owner** badge; your row shows **Member**. The owner-only controls (invite/transfer/dissolve) are no longer available to you, and a **Leave** button now is.

QA-HH-06 — Owner with other members cannot directly leave/dissolve • Edge (Web)

Walkthrough: as owner **with** other members present, confirm the bottom card shows the **Transfer ownership** control (with "you must transfer before leaving" note) — **not** a leave/dissolve button. The single-owner invariant is enforced in the UI.

QA-HH-07 — Sole owner dissolves the household • Core (Web) Automated in CI

Gherkin

```
Given I am the only member and owner of my household
When I click "Leave and delete household" and confirm
Then the household is dissolved and I am re-homed to a fresh solo household
```

Walkthrough

- 1 As sole owner (no other members), bottom card -> **Leave & delete household**.
- 2 **Expected:** a confirm dialog warning the household will be deleted; on confirm, the app reloads and you land signed in with a **new empty household you own** (you're never left tenant-less).

QA-HH-08 — Member leaves the household • Core (Web) Automated in CI

Gherkin

```
Given I am a non-owner member
When I click Leave and confirm
Then I leave the household and am re-homed to a fresh solo household I own
```

Walkthrough

- 1 As a member, **Household** -> **Leave** -> confirm.
- 2 **Expected:** app reloads; you now own a brand-new empty household. The household you left still exists for its remaining members (verify as the owner: the leaver is gone from the member list).

QA-HH-09 — Owner promotes a member to admin • Core (Web) Automated in CI

Precondition: signed in as the owner with at least one **member** present (use the QA-INV flow). **Gherkin**

```
Given I am the owner and another member exists
When I click "Make admin" next to that member
Then their badge changes to Admin
```

Walkthrough

- 1 On **Household**, find a member's row -> click **Make admin**.
- 2 **Expected:** success banner ("Role updated."); the member's badge flips from **Member** to **Admin**, and the button becomes **Make member**. (Verify as that user — see QA-HH-11.)

QA-HH-10 — Owner demotes an admin to member • Core (Web) Automated in CI

Gherkin

```
Given I am the owner and an admin exists
When I click "Make member" next to that admin
Then their badge changes back to Member
```

Walkthrough

- 1 On **Household**, on an **Admin** row -> click **Make member**.
- 2 **Expected:** success banner; the badge returns to **Member** and the button becomes **Make admin**.

QA-HH-11 — Admin sees management controls but not role/ownership controls • Core (Web) Automated in CI

Precondition: signed in as the admin promoted in QA-HH-09. **Gherkin**

```
Given I am an admin (not the owner)
When I open Household
Then I can rename the household and invite/remove members
But I see no promote/demote (role) controls and no transfer/dissolve — only Leave
```

Walkthrough

- 1 As the admin, open **Household**.
- 2 **Expected:** the **rename** field and the **Invitations** card are available; member rows show a **Remove** button but no **Make admin/Make member** buttons (role changes are owner-only). The bottom card shows **Leave** (no Transfer ownership / Leave & delete). The owner's row shows **no** action buttons.

QA-HH-12 — Member sees no management controls • Edge (Web) Automated in CI

Walkthrough: signed in as a plain **member**, open **Household**. **Expected:** read-only name, no **Invitations** card, no per-row action buttons (no **Remove**/role controls), only a **Leave** button — unchanged from QA-HH-02 (a member is never shown management controls regardless of the admin tier).

QA-HH-13 — Owner exports household data • Core (Web)

Gherkin

```
Given I am the household owner on /household
When I click "Download household data"
Then I get a link to a JSON export of the household's data
```

Walkthrough

- 1 As owner, open **Household** -> the **Data** card -> **Download household data**.
- 2 **Expected:** a success row with a **Download** link; following it downloads a JSON bundle containing the tenant, members and invitations (and each feature's data). Members/admins don't see the **Data** card (owner-only). No secrets (invitation token hashes) appear in the file.

QA-HH-14 — Seat quota blocks inviting past the plan limit • Core (Web) Automated in CI

Precondition: the platform ships example seat caps (Free = 3 seats, counting members + pending invites; PlanCatalog). A Free household at its cap (e.g. 3 members, or 2 members + 1 pending invite). **Gherkin**

```
Given my Free plan's seats are all used (members + pending invites)
When I invite another member
Then I'm told my seat limit is reached and to upgrade (nothing is invited)
And raising the limit (Pro plan / editing PlanCatalog) lets the invite through
```

Walkthrough

- 1 As owner of a Free household at the seat cap, **Household** -> invite a new email.
- 2 **Expected:** an error — "Your plan's seat limit is reached. Upgrade your plan to invite more members." (HTTP 402); no invitation is created and no email is sent.
- 3 Free up a seat (revoke a pending invite / remove a member) **or** move to a higher-seat plan — the next invite succeeds. (Seats count members **plus** pending invites, so invites can't over-provision.)
- 4 **Note:** limits are PlanCatalog data; null/absent = unlimited. Metered-usage caps (IQuotaService.TryConsumeAsync, monthly) are wired the same way where an app calls them.

8. Web — Invitations & joining • Core

QA-INV-01 — Owner invites by email; token revealed + email sent • Smoke (Web) Automated in CI

Gherkin

```
Given I am the household owner
When I invite an email address
Then an invitation appears in the pending list with an expiry date
And a one-time join token is revealed in the UI
And an invitation email with a /join link is delivered to Mailpit
```

Walkthrough

- 1 **Household** -> Invitations -> enter `qa.member@gmail.com` -> **Invite**.
- 2 **Expected:** a green panel reveals the raw token + a **Copy** button and notes a join link was emailed; the address shows under **Pending** with an expiry date.
- 3 Check Mailpit: an invitation email addressed to that user, containing a `/join?token=...` link.

QA-INV-02 — Invitee accepts and joins (two-user flow) • Smoke (Web) Automated in CI

Gherkin

```
Given an invitation exists for a second user
When that user opens the /join link, signs in, and accepts
Then they become a member of the inviter's household
```

Walkthrough

- 1 Copy the `/join?token=...` link from QA-INV-01.
- 2 In an **incognito window**, open the link.
- 3 **Expected:** "You've been invited... sign in to accept." Click **Sign in to accept**; complete any sign-in method as `qa.member@gmail.com`.
- 4 **Expected:** after sign-in you're returned to the join flow automatically, it processes, and you see **"You're in"** with a **Go to household** button.
- 5 Click it -> **Household** shows you as a **Member** of "QA House".
- 6 Back in the owner window, reload **Household**: the new member appears and the pending invite is gone.

QA-INV-03 — Already-authenticated user accepts directly • Core (Web)

Walkthrough: while already signed in as a *different* fresh user, open a valid `/join?token=...` **Expected:** it accepts immediately (no sign-in step) and shows success. *(Note: a user already in a household who accepts another invite moves to the new household — verify their old membership is replaced, honoring the one-tenant invariant.)*

QA-INV-04 — Join link with missing token • Edge (Web)

Walkthrough: open `/join` with no `?token=`. **Expected:** "invalid invitation / missing token" state, no crash.

QA-INV-05 — Join with invalid/expired/used token • Core (Web)

Gherkin

```
Given an invitation token that is invalid, already used, or expired
When an authenticated user opens its /join link
Then they see an error state, not a successful join
```

Walkthrough: while signed in, open `/join?token=garbage123` (or reuse a token already accepted in QA-INV-02). **Expected:** the error state with a **Back to household** link; no membership change.

QA-INV-06 — Invite an existing member is rejected • Core (Web)

Gherkin

```
Given a user is already a member of my household
When I invite their email again
Then I get an "already a member" error and no duplicate invite is created
```

Walkthrough: as owner, invite the email of someone already in the household. **Expected:** a red "already a member" status (HTTP 409); nothing added to pending.

QA-INV-07 — Regenerate a pending invitation • Core (Web)

Gherkin

```
Given a pending invitation exists
When I click Regenerate
Then a new token is issued (revealed) and the old token no longer works
```

Walkthrough

- 1 On a pending invite, click **Regenerate**.
- 2 **Expected:** a new token is revealed. The **previous** token's `/join` link now fails (QA-INV-05), the new one works.

QA-INV-08 — Revoke a pending invitation • Core (Web) Automated in CI

Gherkin

```
Given a pending invitation exists
When I click Revoke
Then it is removed from pending and its token no longer works
```

Walkthrough: click **Revoke** on a pending invite. **Expected:** "invitation revoked" status; it leaves the pending list; opening its old link gives the error state.

QA-INV-09 — Copy token button • Edge (Web)

Walkthrough: click **Copy** on a revealed token; paste elsewhere. **Expected:** the token is on the clipboard. (If the browser blocks clipboard access, the token is still visible to copy manually — no error shown.)

9. Web — Settings / linked accounts • Core

QA-SET-01 — View linked providers • Core (Web)

Gherkin

```
Given I signed up with Google
When I open /settings
Then Google shows as "Connected" and Microsoft shows a "Link" button
```

Walkthrough: sign in with Google, open **Settings**. **Expected:** a row per provider; the one you used shows a **Connected** badge + **Unlink**; the other shows **Link**.

QA-SET-02 — Link a second provider • Core (Web)

Gherkin

```
Given I am signed in and Microsoft is not linked
When I click Link on Microsoft and complete consent
Then Microsoft becomes Connected on my account (no new account is created)
```

Walkthrough

- 1 **Settings** -> **Link** on Microsoft -> complete consent with a Microsoft account **whose email isn't already used by another app user**.
- 2 **Expected:** returned to Settings with a success banner ("Microsoft linked"); Microsoft now shows **Connected**. You can subsequently sign in with either provider into the **same** account.

QA-SET-03 — Linking a provider already used by another account is rejected • Core (Web)

Gherkin

```
Given a provider identity is already linked to a different user
When I try to link it to my account
Then I get an "already in use" error and it is not linked
```

Walkthrough: try to **Link** a Google/Microsoft identity that another app user already owns. **Expected:** Settings shows an "already in use" banner (?link_error=in_use); no link made. *(Expired link-token path shows ?link_error=expired — exercise if you can stall the flow past the token lifetime.)*

QA-SET-04 — Unlink a provider • Core (Web)

Gherkin

```
Given two providers are linked to my account
When I click Unlink on one and confirm the dialog
Then it returns to a "Link" state
```

Walkthrough: with two providers connected, click **Unlink** on one. **Expected:** a confirm dialog ("Unlink this sign-in method?"); on **confirm** the row reverts to **Link**. **Cancelling** the dialog leaves it **Connected** — no change. *(The confirm fails closed: if the browser dialog can't run, the unlink is cancelled, never silently performed. The same guarded confirm protects remove-member / leave / dissolve in §7.)*

QA-SET-05 — Unlinking your only provider never locks you out • Edge (Web)

Gherkin

```
Given Google is my only linked provider
When I unlink it
Then I can still sign in by email (magic link / OTP)
```

Walkthrough: unlink your sole provider, sign out, and sign back in with **OTP/magic link** using the same email.

Expected: you reach the **same** account/household. (Email sign-in is always available by design, so provider removal can't lock you out.)

QA-SET-06 — Settings requires auth • Edge (Web)

Walkthrough: signed out, navigate to /settings. **Expected:** redirect to /login.

QA-SET-07 — Delete my account • Core (Web) Automated in CI

Precondition: use a **throwaway** account (this is destructive). Easiest: a member of another owner's household (so no dissolve). **Gherkin**

```
Given I am signed in on /settings
When I use the Danger zone "Delete my account" and confirm
Then my account and personal data are deleted and I'm signed out
```

Walkthrough

- 1 **Settings** -> **Danger zone** -> **Delete my account** -> confirm the dialog.
- 2 **Expected (member):** account deleted; you're signed out and land on /login. Signing in again creates a brand-new account.
- 3 **Owner with other members:** an error tells you to **transfer ownership first** (nothing deleted).
- 4 **Sole owner:** a second confirm warns it also **dissolves the household**; on confirm, the account + household are deleted.

QA-MFA-01 — Enable two-factor (authenticator TOTP) • Core (Web) Automated in CI

Precondition: signed in; an authenticator app (Google Authenticator, 1Password, Authy, ...) to hand. **Gherkin**

```
Given I am signed in on /settings with two-factor Off
When I enable it, scan the QR (or enter the key) and confirm with a 6-digit code
Then two-factor turns On and I'm shown one-time recovery codes
```

Walkthrough

- 1 **Settings** -> **Two-factor authentication** shows an **Off** badge -> **Enable two-factor**.
- 2 A **QR code** renders next to a **manual key** (Base32). Scan it (or type the key) into the app.

- 3 Enter the app's current **6-digit code** -> **Verify & enable**.
- 4 **Expected:** a success banner, the badge flips to **On**, and a grid of **recovery codes** appears (shown once). **I've saved my codes** returns to the On state.
- 5 **Wrong code:** an inline error ("that code is incorrect or has expired"); nothing changes.

QA-MFA-02 — Two-factor is required at sign-in • Core (Web) Automated in CI

Precondition: an account with two-factor **On** (QA-MFA-01). **Gherkin**

```
Given my account has two-factor enabled
When I sign in with an email OTP
Then I'm asked for an authenticator code before the session starts
And entering a valid code completes sign-in
```

Walkthrough

- 1 Sign out. On /login, request an **email code**, enter it.
- 2 **Expected:** instead of landing signed-in, a **second prompt** asks for the authenticator code.
- 3 Enter the current 6-digit code -> you're signed in (lands on /).
- 4 **Wrong/expired code:** an inline error; you stay on the step-up prompt (no session).
- 5 **Note:** OAuth and magic-link sign-ins enforce the step-up too (QA-MFA-04); native (MAUI) too (QA-MFA-05).

QA-MFA-03 — Use a recovery code, then disable two-factor • Core (Web)

Gherkin

```
Given my account has two-factor enabled
When I sign in and enter a recovery code at the step-up
Then sign-in completes (that code is now spent)
And I can disable two-factor from Settings with a valid code
```

Walkthrough

- 1 Sign in as in QA-MFA-02; at the step-up, enter one **recovery code** instead of a TOTP -> signs in.
- 2 **Settings** -> **Two-factor authentication** (On) -> **Disable two-factor** -> enter a current TOTP (or another recovery code) -> **Disable**.
- 3 **Expected:** the badge flips to **Off**; a subsequent sign-in no longer asks for a second step.

QA-MFA-04 — Redirect logins (OAuth / magic link) enforce the step-up • Core (Web)

Precondition: an account with two-factor **On**, reachable via an OAuth provider and/or magic link. **Gherkin**

```
Given my account has two-factor enabled
When I sign in with Google/Microsoft or a magic link
Then I am redirected to the authenticator step-up before any session is issued
And entering a valid code completes sign-in
```

Walkthrough

- 1 Sign out. Sign in via **Google/Microsoft** (or click a **magic link**).
- 2 **Expected:** instead of landing signed-in, you arrive on /login showing the **authenticator code prompt** (the URL carries a one-time ?mfa= challenge). No session exists yet.
- 3 Enter the current 6-digit code (or a recovery code) -> you're signed in (/auth-callback -> /).
- 4 **Security check:** confirm you are **not** signed in until the code is accepted — a wrong/expired code keeps you on the prompt with no session. (This closes the gap where redirect logins skipped MFA.)

QA-MFA-05 — Native (MAUI) sign-in enforces the step-up • Core (Desktop/Android)

Precondition: an account with two-factor **On**; run the desktop/Android shell (see docs/MOBILE_TESTING.md). Native has no magic link — use **email OTP** or **OAuth**. **Gherkin**

```
Given my account has two-factor enabled
When I sign in on the native app with an email code or OAuth
Then the app shows the authenticator step-up in-app before completing sign-in
And a valid code (or recovery code) finishes sign-in
```


Walkthrough

- 1 In the native app, sign in with an **email code** (or a provider).
- 2 **Expected:** the app stays on the login screen and shows the **authenticator code prompt** (it does not sign in yet). No session/token is stored.
- 3 Enter the current 6-digit code (or a recovery code) -> you're signed in (lands on /).
- 4 **Wrong/expired code:** inline error; you remain on the prompt (no session). Tokens arrive in the response body (native transport), same as a normal native login.

QA-NOTIF-01 — Notification bell + list • Core (Web) Automated in CI

Note: the platform has no built-in producer; to see items, a feature must call `INotificationService.NotifyAsync` (seed one in dev, or exercise a downstream feature that notifies). **Gherkin**

```
Given I am signed in
When I open the notification bell in the header
Then I see my notifications newest-first, with unread ones marked
And the bell shows an unread count when I have unread notifications
```

Walkthrough

- 1 In the header, click the **bell**. **Expected:** a dropdown opens; with none, it reads "You're all caught up."
- 2 With unread notifications present: a red **count badge** shows on the bell; unread rows carry a dot + bold title; each shows a relative time ("just now", "5m", "3h", "2d").
- 3 Click the backdrop (outside the panel) -> it closes.

QA-NOTIF-02 — Mark read / mark all read • Core (Web) Automated in CI

Precondition: at least one unread notification (see QA-NOTIF-01 note). **Gherkin**

```
Given I have unread notifications
When I click one (and, separately, "Mark all read")
Then that one clears its unread mark and the count drops
And "Mark all read" zeroes the count
```

Walkthrough

- 1 Open the bell -> click an **unread** row. **Expected:** its dot/bold clears; the count decrements.
- 2 Click **Mark all read**. **Expected:** the count badge disappears; all rows show as read.
- 3 Reload the page -> the counts/read state persist (server-side).

QA-NOTIF-03 — Delivery preferences • Core (Web) Automated in CI

Gherkin

```
Given I am signed in on /settings
When I toggle the In-app or Email notification switches
Then the choice is saved and survives a reload
```

Walkthrough

- 1 **Settings** -> **Notifications** card shows two switches (**In-app**, **Email**), both on by default.
- 2 Toggle one off. **Expected:** it saves immediately (optimistic; reverts with an error if it fails).
- 3 Reload -> the switch keeps its new state. (Email-off suppresses the email channel on future notifies; in-app-off suppresses the in-app row.)

10. Web — Localization (i18n) • Core

QA-I18N-01 — Switch language on the login page • Core (Web) Automated in CI

Gherkin

```
Given I am on /login in English
When I choose Español in the language switcher
Then the page text renders in Spanish
```

Walkthrough: on `/login`, use the language switcher (bottom of the card) -> **Español**. **Expected:** titles, button labels, and prompts switch to Spanish; the choice persists on reload.

QA-I18N-02 — Language persists per user across sessions • Core (Web)

Gherkin

```
Given I am signed in and set my language to Spanish
When I sign out and sign back in (even in a fresh browser)
Then the app loads in Spanish
```

Walkthrough

- 1 Signed in, switch to **Español**. (This saves the locale to your user record via `PUT /api/auth/locale` and into the JWT.)
- 2 Sign out; sign back in.
- 3 **Expected:** app comes up in Spanish — the preference followed the `*user*`, not just the browser.

QA-I18N-03 — In-app UI is fully translated (no English leaks) • Edge (Web)

Walkthrough: in Spanish, walk Home -> Household -> Settings -> invite flow. **Expected:** all visible labels/buttons/validation/status messages are Spanish; flag any English string that leaks.

QA-I18N-04 — Email language matches the requester's UI language • Core (Web)

Gherkin

```
Given my UI language is Spanish
When I trigger an OTP / magic link / invitation email
Then the email arrives in Spanish
```

Walkthrough

- 1 Set UI to **Español**. Trigger an OTP (and a magic link, and send an invite).
- 2 In Mailpit, open each. **Expected:** subject + body in Spanish. Repeat in English to confirm both.

10b. Web — Admin console (platform staff) • Core

Precondition: your account's email must be in the staff allowlist — set `Admin__StaffEmails__0` in the repo-root `.env` (see `.env.example`) and restart the API. Non-staff accounts must **not** see any of this.

QA-ADMIN-01 — Staff sees the console; non-staff don't • Core (Web) Automated in CI

Gherkin

```
Given I am signed in as a platform-staff user
When I look at the header
Then I see an "Admin" link, and /admin lists every tenant with member counts
And a non-staff user sees no Admin link and is refused at /admin
```

Walkthrough

- 1 Signed in as **staff**: an **Admin** button shows in the header -> open it (or go to `/admin`).
- 2 **Expected:** the **Tenants** list shows every tenant (name + member count). Click one -> detail panel shows members (name/email + role), subscription status, created date, audit-event count.
- 3 Sign in as a **non-staff** user: **no Admin link**; navigating directly to `/admin` shows "You don't have access to the admin console."

Automated: staff console + tenant list (announcement journey) and the non-staff `/admin` refusal.
The header **Admin-link visibility** checks (steps 1 and 3's "no Admin link") remain manual.

QA-ADMIN-02 — View a tenant is audited in that tenant • Core (Web)

Gherkin

```
Given I am staff viewing a tenant's detail in /admin
When the detail loads
Then an audit event (admin.tenant.viewed) is recorded in that tenant
```

Walkthrough

- 1 As staff, open a tenant's detail in /admin.
- 2 **Expected:** an `admin.tenant.viewed` event is written in **that tenant** (visible to that tenant's own audit trail) — the global tenant filter is never loosened; the read enters the target tenant.

QA-ADMIN-03 — Impersonate a user, then stop • Core (Web)

Gherkin

```
Given I am staff on a tenant's detail
When I "Sign in as" a member and confirm
Then I browse the app as that user with a persistent impersonation banner
And "Stop impersonating" returns me to my own staff identity
```

Walkthrough

- 1 In a tenant detail, click **Sign in as** on a member -> confirm the dialog.
- 2 **Expected:** you land on / **as that user** (their name/household in the header); a yellow **impersonation banner** is pinned at the top; the **Admin** link is hidden while impersonating.
- 3 Click **Stop impersonating**. **Expected:** you're back as yourself (staff); the banner is gone.
- 4 The impersonation token is **short-lived (15 min) and non-refreshable** — a full page reload also returns you to your own identity. Impersonation is **audited** in the target's tenant.

QA-ADMIN-04 — Staff announcement reaches a tenant's members • Edge (Web) Automated in CI

Gherkin

```
Given I am staff on a tenant's detail in /admin
When I send an announcement (title + body) and confirm
Then every member of that tenant is notified through their preferred channels
And each member's bell shows the announcement; reading it clears the badge
And an admin.announcement.sent audit event is recorded in that tenant
```

Walkthrough

- 1 As staff, open a tenant's detail in /admin -> fill **Send announcement** (title + message) -> **Send to all members** -> confirm. **Expected:** "Announcement sent to N member(s)."
- 2 Sign in as a member of that tenant. **Expected:** the bell badge shows the unread announcement; opening it shows title + message; clicking it marks it read and the badge clears. Members with email delivery on also get the email (Mailpit).
- 3 The send is audited in **that tenant** (`admin.announcement.sent`, with the member count).

10c. Web — Billing page • Core

With no Stripe keys configured (dev default) the **FakeBillingProvider** is active: checkout/portal return `https://billing.test/...` URLs (a dead domain — expected), and the webhook accepts the signature `valid`. With real Stripe test keys, use Stripe's hosted test checkout instead.

QA-BILL-01 — Billing page shows plan + seats; owner-only • Edge (Web) Automated in CI

Gherkin

```
Given I am the household owner on /billing
Then I see my current plan, its status, and seat usage vs the plan limit
And a member opening /billing sees only the "ask your owner" notice
```

Walkthrough

- 1 As **owner**, open **Billing** in the header. **Expected:** current plan (e.g. free), status badge, and seats (e.g. 1 of 3 used); an **Upgrade to Pro** button on the free plan; **Manage subscription** only when a subscription exists.
- 2 As a **member**, open `/billing`. **Expected:** no plan/usage — just the owner-only notice.

QA-BILL-02 — Upgrade via checkout + provider webhook lands on Pro • Core (Web) Automated in CI

Gherkin

```
Given the owner on the free plan clicks Upgrade to Pro
When the checkout redirect fires and the provider webhook lands (subscription active)
Then /billing shows plan pro, status active, the pro seat limit, and the portal button
```

Walkthrough

- 1 Click **Upgrade to Pro**. **Expected:** redirect to the provider checkout (fake: `billing.test/checkout/{tenantId}/pro` — the page won't load, which is fine).
- 2 Simulate payment completion by POSTing the webhook (fake provider): `POST {api}/api/billing/webhook` with header `Stripe-Signature: valid` and a PascalCase JSON body (`EventId`, `TenantId`, `PlanKey: "pro"`, `Status: "active"`, `StripeCustomerId`, `OccurredAt`). **Expected:** 200.
- 3 Reload `/billing`. **Expected:** plan pro / status active, seats x of 10, **Manage subscription** now visible.

11. Emails (Mailpit) — branding & content • Core

Delivery is asynchronous (the outbox dispatcher) — emails land in Mailpit a few seconds after the triggering action, and the send request succeeds regardless (see §1.1). Wait briefly before opening Mailpit; a missing email is a delayed/retrying/dead-lettered outbox message, not a request failure.

QA-MAIL-01 — OTP email is branded & correct • Core

Gherkin

```
Given I request an OTP code
When I open the email in Mailpit
Then it shows the brand logo, brand colours, the 6-digit code, and the tagline
```

Walkthrough: trigger an OTP; open it in Mailpit. **Expected:** the logo image renders (CID-embedded, not a broken image), brand-green styling, a clearly displayed code, footer wordmark + tagline.

QA-MAIL-02 — Magic-link email • Core

Walkthrough: trigger a magic link; open in Mailpit. **Expected:** branded layout; a working sign-in button/link; sensible subject.

QA-MAIL-03 — Invitation email • Core

Walkthrough: send an invite; open in Mailpit. **Expected:** branded layout; the recipient's address; a `/join?token=...` link that works (QA-INV-02).

QA-MAIL-04 — Logo renders in a real client • Edge

Walkthrough (optional, deliverability): forward/inspect one email in a real Gmail/Outlook client. **Expected:** the CID logo still renders. *(Note: from a non-verified domain, real delivery may land in spam — a domain/DKIM concern, not an app bug.)*

12. Desktop — MAUI / Windows • Core

The desktop client reuses the same UI; only the **auth transport** (loopback browser flow, body token, OS secure storage) differs. Magic link is intentionally **absent** on native.

QA-DSK-01 — OTP sign-in • Smoke (Desktop) Automated in CI

Gherkin

```
Given the desktop app is on the login screen
When I request an OTP and enter the code from Mailpit
Then I am signed in within the app (no browser)
```

Walkthrough

- 1 Launch the desktop app -> login screen. **Expected:** Google/Microsoft buttons + email field with **Email me a code**; **no magic-link button** (native hides it).
- 2 Enter an email -> request code -> get it from Mailpit -> enter it.
- 3 **Expected:** signed in, app shell shown, household + name in header.

QA-DSK-02 — Google OAuth via loopback • Core (Desktop)

Gherkin

```
Given the desktop login screen
When I click Continue with Google and complete consent in the system browser
Then the browser shows a "you can close this" page and the app completes sign-in
```

Walkthrough

- 1 Click **Continue with Google**. **Expected:** your **system browser** opens to Google consent.
- 2 Approve. **Expected:** the browser tab shows a "you can close this tab" page; **focus returns to the app**, now signed in. (Repeat for **Microsoft**.)

QA-DSK-03 — "Remember me" across app restart • Core (Desktop)

Gherkin

```
Given I am signed in to the desktop app
When I fully close and reopen it
Then I am still signed in
```

Walkthrough: close the app completely; relaunch. **Expected:** lands signed in (refresh token held in Windows secure storage / DPAPI, silently exchanged on startup).

QA-DSK-04 — Household & Settings load (authorized API calls) • Core (Desktop)

Gherkin

```
Given I am signed in to the desktop app
When I open Household and Settings
Then both load their data without an auth error
```

Walkthrough: open **Household** (members load) and **Settings** (linked providers load). **Expected:** both populate — no spinner-forever, no "couldn't load" error. *(This is the Bearer-token-handler path that previously failed on native; it must work.)*

QA-DSK-05 — Link a provider from Settings • Edge (Desktop)

Walkthrough: **Settings** -> **Link** Microsoft -> system browser loopback flow -> returns to app. **Expected:** Microsoft shows **Connected**; the app shell was never navigated away.

QA-DSK-06 — Sign out • Edge (Desktop)

Walkthrough: **Sign out**. **Expected:** back to the login screen; reopening the app does **not** auto-sign-in (secure storage cleared).

QA-DSK-07 — Core household flows work on desktop • Edge (Desktop)

Walkthrough: spot-check rename, invite (token revealed), and leave on desktop. **Expected:** same behavior as web (§7–8) — the UI is shared.

****NATIVE Wave 2 (ADR-018, docs/NATIVE_PARITY.md):**** the cases below verify the parity fixes per-feature on desktop — join-by-code (G5), culture persistence (G6), export share (G1), and the billing return-trip (G2) — plus the feature areas the old plan never exercised natively.

QA-DSK-08 — Join a household by pasted invite code • Core (Desktop)

Gherkin

```
Given a member account signed in on desktop and an invite code from the owner
When I open Household -> "Have an invite?" and paste the code
Then I join the owner's household exactly as the emailed link would
```

Walkthrough

- 1 As the **owner** (web is fine): Household -> invite the member's email -> copy the revealed token.
- 2 On **desktop** as the member: **Household** -> **Have an invite?** -> paste the code -> **Join**. **Expected:** success state -> **Go to household** -> roster lists both members.
- 3 Negative: paste a garbage code. **Expected:** inline error, the form stays for a retry.

QA-DSK-09 — Language choice survives an app restart • Core (Desktop)

Gherkin

```
Given the desktop app on the login screen (signed out)
When I switch the language to Español and fully restart the app
Then it launches in Spanish
```

Walkthrough

- 1 Signed out, on the login screen: switch the language selector to **Español**. **Expected:** the UI re-renders in Spanish (no restart needed).
- 2 Fully close the app; relaunch. **Expected:** still Spanish — the choice is read from OS Preferences before first render (NATIVE-5). Switch back to English; same persistence.
- 3 Note: after sign-in, a **server-saved** locale wins (the account preference follows you across devices) — that's by design.

QA-DSK-10 — Data export downloads via the OS • Core (Desktop)

Gherkin

```
Given I am the owner, signed in on desktop
When I request the data export and click Download
Then the file is offered through the platform share/save UI, not a dead WebView navigation
```

Walkthrough

- 1 **Household** -> **Data** -> **Export my data** -> wait for the ready alert -> **Download**.
- 2 **Expected:** the **Windows share flyout** opens with the JSON bundle staged (server-named ...-<id>.json); the app page is not navigated away. Save it and open — valid JSON, no secrets (spot-check: no token hashes).

QA-DSK-11 — Billing: checkout leaves, summary refreshes on return • Core (Desktop)

Gherkin

```
Given I am the owner on the desktop Billing page (Stripe test mode configured)
When I click Upgrade, complete checkout in the system browser, and return to the app
Then the plan summary refreshes by itself – no re-navigation needed
```

Walkthrough

- 1 **Billing** -> **Upgrade**. **Expected:** the **system browser** opens the hosted checkout; the app stays on Billing (it does not navigate away).
- 2 Complete checkout with a test card. **Expected:** the browser lands on the **web** app's billing page (by design — emailed/redirect links are web; docs/NATIVE_PARITY.md G2).
- 3 Click back into the desktop app. **Expected:** the summary **refetches on focus** — the plan flips to **pro** without touching navigation. (Without Stripe keys the fake provider's checkout URL is a stub — then verify only: browser opened, app stayed, and refocusing refetches.)

QA-DSK-12 — MFA: enroll and native step-up • Core (Desktop)

Gherkin

Given I am signed in on desktop without MFA
When I enable the authenticator in Settings, sign out, and sign in again
Then the app itself prompts for the 6-digit code before completing sign-in

Walkthrough

- 1 **Settings** -> **Two-factor** -> **Enable**. **Expected:** the QR renders inside the app (the vendored QR script ships in the native host too); manual key shown; confirm with a code; **recovery codes** displayed once.
- 2 Sign out -> OTP sign-in. **Expected:** after the OTP, the app shows the **in-app MFA prompt** (native step-up, MFA-4) — enter the authenticator code -> signed in. A recovery code also works (single-use).

QA-DSK-13 — Notification bell & preferences • Edge (Desktop)

Walkthrough: header **bell** -> **Expected:** opens with the empty state (or your items), badge matches unread count.

Settings -> notification preferences -> toggle one off -> reload the page -> **Expected:** the switch state persisted. (For a real item end-to-end, a staff announcement — see QA-ADMIN-04 — lands in the native bell too; spot-check when staff is configured.)

QA-DSK-14 — Admin console on desktop (staff only) • Edge (Desktop)

Walkthrough: with your email in Admin:StaffEmails, sign in on desktop. **Expected:** Admin appears in the nav; the console lists tenants; opening a detail works. Impersonate a user -> **Expected:** banner appears; **Stop** restores your staff session in-app (native refresh path).

13. Android — MAUI • Core

Prereq every run: `**adb reverse tcp:5238 tcp:5238**` + API on the https profile. See docs/MOBILE_TESTING.md.

QA-AND-01 — OTP sign-in • Smoke (Android) Automated in CI

Gherkin

Given the Android app is on the login screen with adb reverse set
When I request an OTP and enter the code from Mailpit
Then I am signed in

Walkthrough

- 1 Confirm `adb reverse --list` shows `tcp:5238`. Launch the app.
- 2 Login screen: email field + **Email me a 6-digit code**; Google/Microsoft buttons; **no magic link**.
- 3 Enter an email -> request code -> read it in Mailpit (host) -> enter it.
- 4 **Expected:** signed in, app shell shown.

QA-AND-02 — Google OAuth via custom scheme • Core (Android)

Gherkin

Given the Android login screen
When I tap Continue with Google and complete consent
Then the in-app browser returns to the app via the `perezosoftware://` scheme, signed in

Walkthrough

- 1 Tap **Continue with Google**. **Expected:** a browser tab opens to Google consent.
- 2 Approve. **Expected:** the tab returns control to the app (`perezosoftware://auth` intent), now signed in. (Repeat **Microsoft** if its `:5238` redirect is registered.)

QA-AND-03 — "Remember me" across app restart • Core (Android)

Walkthrough: swipe-close the app; reopen. **Expected:** still signed in (refresh token in the Android Keystore). *(If `adb reverse` was lost on a device reboot, re-run it first — a startup failure after reboot is an environment issue, not an app bug.)*

QA-AND-04 — Household & Settings load • Core (Android)

Walkthrough: open **Household** and **Settings**. **Expected:** both load their data (the native Bearer path works), same as desktop QA-DSK-04.

QA-AND-05 — Navigation drawer / header is reachable • Edge (Android)

Gherkin

```
Given I am signed in on Android
When I open the navigation (hamburger)
Then the menu is tappable and not hidden under the status bar
```

Walkthrough: tap the hamburger; use **Household/Settings/Sign out**. **Expected:** the header sits below the status bar (safe-area padding) and every item is tappable.

QA-AND-06 — Core flows on Android • Edge (Android)

Walkthrough: spot-check language switch, invite (token revealed), and leave. **Expected:** parity with web.

****NATIVE Wave 2 (ADR-018, docs/NATIVE_PARITY.md):**** the cases below verify the parity fixes on Android — including the two that only exist on this platform (hardware back, share sheet).

QA-AND-07 — Hardware back navigates in-app history • Smoke (Android)

Gherkin

```
Given I am signed in on Android and have navigated Home -> Household -> Settings
When I press the hardware/gesture back button repeatedly
Then it walks back through the app's pages and only leaves the app at the root
```

Walkthrough

- 1 Navigate **Home -> Household -> Settings** (three distinct pages).
- 2 Press **back**. **Expected:** Settings -> Household. Again: -> Home.
- 3 Press **back at Home (root)**. **Expected:** the app backgrounds/exits — the default, but only at the root. *(Before NATIVE-4, any back press exited the app.)*
- 4 Note: a full WebView reload (e.g. the language switch) restarts the in-page history — back exiting right after one is acceptable.

QA-AND-08 — Join a household by pasted invite code • Core (Android)

Walkthrough: as QA-DSK-08, on Android: owner invites (web) -> member on Android: **Household -> Have an invite?** -> paste -> **Join** -> roster shows both. Garbage code -> inline error, retryable. *(This was parity gap G5 — a native member previously had no way to join at all.)*

QA-AND-09 — Language choice survives an app restart • Core (Android)

Walkthrough: as QA-DSK-09: signed out, switch to **Español** (re-renders) -> **swipe-close** the app -> relaunch. **Expected:** still Spanish (OS Preferences bootstrap, NATIVE-5). Signed-in accounts reconcile to their server-saved locale — by design.

QA-AND-10 — Data export via the share sheet • Core (Android)

Walkthrough: owner -> **Household -> Data -> Export my data -> Download**. **Expected:** the **Android share sheet** opens with the JSON bundle (server-named); share to Files/Drive and open — valid JSON. The app page is not navigated away. *(Before NATIVE-3 this click did nothing — the WebView silently dropped it.)*

QA-AND-11 — Billing: checkout leaves, summary refreshes on return • Core (Android)

Walkthrough: as QA-DSK-11: **Billing -> Upgrade -> Expected:** the system browser/Custom Tab opens; complete checkout (test mode); switch back to the app (app switcher or back). **Expected:** the summary **refetches on resume** — plan shows **pro** without re-navigation (NATIVE-4; Android's return path is the Activity resume).

QA-AND-12 — MFA: enroll and native step-up • Core (Android)

Walkthrough: as QA-DSK-12 on Android — QR renders in-app (scan it with a second device or use the manual key), recovery codes shown once; after sign-out, OTP sign-in prompts the **in-app** MFA step-up (MFA-4).

QA-AND-13 — Edge-to-edge / status bar on Android 15 • Edge (Android)

Gherkin

```
Given a device or emulator on Android 15 (API 35, edge-to-edge enforced)
When I use the app in portrait and landscape
Then no control is hidden under the status bar or gesture areas
```

Walkthrough: check the header/hamburger (extends QA-AND-05), the bell dropdown, and a page with bottom-of-screen buttons (Settings danger zone) in both orientations. **Expected:** nothing sits under the status bar or the gesture-nav pill; everything tappable. *(Flagged [verify] by the parity audit — if this fails, it becomes a small safe-area fix slice.)*

13b. iOS + macCatalyst — first-run smoke • Core

These platforms compile in CI but had never been RUN before this pass. Prereqs: a Mac with Xcode 26.5 (the CI pin), the repo, and the API + Postgres + Mailpit running on that Mac (the iOS simulator shares the host network, so `https://localhost:7160` works; a **physical device** needs the API bound to a LAN address + the dev cert trusted). Launch: `dotnet build src/Maui -t:Run -f net10.0-ios (simulator) / -f net10.0-maccatalyst`. The **G7 fix is required** (PR #109) — before it, both platforms crashed at first resolve (no IOAuthInitiator registered). OAuth is wired via ASWebAuthenticationSession + the perezosoft scheme in Info.plist but has **never been exercised** — treat QA-IOS-04 as its first real test.

QA-IOS-01 — App boots to the login screen • Smoke (iOS)

Walkthrough: launch on the simulator. **Expected:** the app opens (no startup crash — this **was** parity gap G7), the login screen renders: email + **Email me a 6-digit code**, provider buttons, **no magic-link button**, nothing under the notch/safe areas.

QA-IOS-02 — OTP sign-in • Smoke (iOS)

Walkthrough: request a code -> read it from Mailpit (on the Mac) -> enter it. **Expected:** signed in, app shell + household load (the native Bearer/body-token path works on iOS).

QA-IOS-03 — Core flows spot-check • Core (iOS)

Walkthrough: spot-check on the simulator: **Household** roster + invite (token revealed) + **join-by-code**, **Settings** (providers list, MFA card renders its QR), **language switch** -> re-render + relaunch persistence, **bell** empty state, **export** -> iOS share sheet. **Expected:** parity with Android (§13) — same shared RCL.

QA-IOS-04 — OAuth via ASWebAuthenticationSession • Core (iOS)

Walkthrough: **Continue with Google**. **Expected:** the system auth session sheet opens to Google consent; approving returns to the app via the `perezosoft://auth` scheme, signed in. Also verify session-across-restart (Keychain-backed secure storage).

QA-MAC-01 — App launches at a usable window • Smoke (macCatalyst)

Walkthrough: launch the macCatalyst build. **Expected:** no startup crash (G7); the window opens at a sensible default size and is resizable; login renders. *(The audit's [verify] desktop-window-sizing cell for macOS.)*

QA-MAC-02 — OTP sign-in • Smoke (macCatalyst)

Walkthrough: as QA-IOS-02. **Expected:** signed in; Household + Settings load.

QA-MAC-03 — Core flows + OAuth spot-check • Core (macCatalyst)

Walkthrough: as QA-IOS-03 (share = macOS share menu) + one OAuth round-trip (QA-IOS-04 flow) + restart persistence. **Expected:** parity.

13c. Native release checklist

Per release that ships a native client, run the platform's • **Smoke** cases plus **one** • **Core** feature spot-check, and record results in §16:

| Platform | Always (• Smoke) | Plus one of (• Core) |
|-----------------|--------------------------|------------------------------|
| Windows desktop | DSK-01, DSK-03, DSK-06 | DSK-08..12 |
| Android | AND-01, AND-03, AND-07 | AND-08..12 |
| iOS | IOS-01, IOS-02 | IOS-03, IOS-04 |
| macCatalyst | MAC-01, MAC-02 | MAC-03 |

Full per-feature native regression (every case in §12–13b) is for releases that changed native glue (`src/Mauui/**`, the RCL seams: `ICulturePersistence` / `IFileDownloadLauncher` / `AppResumeNotifier`) or bumped the .NET/MAUI toolchain.

14. Cross-cutting security • **Core**

QA-SEC-01 — Tenant isolation • **Smoke**

Gherkin

```
Given two households owned by two different users
When user A is signed in
Then A can only ever see A's household, members, and invitations – never B's
```

Walkthrough

- 1 Create household A (owner A) and a separate household B (owner B, different account/incognito).
- 2 As A, open **Household**. **Expected:** only A's members/invites. Confirm B's data never appears. *(Deeper API-level probing — e.g. requesting another tenant's resource id — belongs in `automated Api.Tests`; this manual case verifies the UI never cross-contaminates.)*

QA-SEC-02 — Protected pages require auth • **Core**

Walkthrough: signed out, directly visit `/household`, `/settings`. **Expected:** each redirects to `/login`.

QA-SEC-03 — Session is gone after sign-out • **Core**

Gherkin

```
Given I sign out
When I press the browser Back button or reload a protected page
Then I am not able to access it – I am sent to /login
```

Walkthrough: sign out, press **Back** to a protected page / reload it. **Expected:** bounced to `/login`; no stale authenticated view.

QA-SEC-04 — Native open-redirect guard • **Edge (Desktop/Android)**

Context/Expected: the native OAuth flow only honors loopback http callbacks or the configured `perezosoftware://` scheme; arbitrary redirect targets are rejected. This is unit-tested (`NativeRedirectPolicyTests`); no manual action needed unless probing the API directly — record as **covered by automated tests**.

QA-SEC-05 — Server-side hardening (automated) • **Edge**

Context/Expected: several invariants are enforced at the API/data layer and verified by `tests/Api.Tests`, not by manual UI steps — record as **covered by automated tests** unless probing the API directly:

- **Tenant write-stamping** — a new tenant-scoped row is stamped with the caller's tenant and a foreign-tenant write is rejected (`TenantStampingInterceptorTests`), so reads *and* writes are tenant-isolated.
- **Refresh-token reuse detection** — replaying an already-rotated refresh token revokes all the user's sessions (`RefreshTokenServiceTests`). Manually observable only by capturing and replaying a refresh cookie/token; out of scope for routine QA.
- **Unverified-email takeover guard** fails closed (`ClaimsExtractorTests` / `UserServiceTests`).

- **Legacy refresh-cookie self-heal** — a stale Path=/ refresh cookie left by an older build can shadow the live Path=/api/auth cookie and wedge sign-in into a /refresh 401 -> "Authentication Failed" loop (seen in Firefox, due to cookie send-order). Setting the refresh cookie now also emits an expiry for the orphan, so the next successful sign-in / refresh sweeps it automatically — no manual cookie-clearing, and upgraded deployments self-heal (CookieServiceTests). *Manual repro needs a planted Path=/ cookie; record as covered by automated tests. If a tester hits a stuck /refresh 401 loop in Firefox after a deploy, a single re-sign-in clears it.*

14b. API surfaces — PUBAPI + HOOKS (config-gated; curl / Postman) • Core

These two surfaces are **off by default** and have **no web UI** by design — they're for machines, so they're QA'd with an HTTP client. Any client works; the steps use `curl`. **Postman**: import `GET /api/public/openapi.json` (once PUBAPI is on) to get a ready collection for the public routes.

Preconditions (do once): in the repo-root `.env` set `PublicApi__Enabled=true` and `Webhooks__Enabled=true`, then restart the API. Management of keys/webhooks is **owner-only**, so sign in as an owner and grab a JWT access token from `POST /api/auth/refresh` (the Swagger "Authorize" button shows one) to call the `/api/apikeys` and `/api/webhooks` management routes below. Base URL in these steps is `https://localhost:7160` (use `-k` for the dev cert).

QA-API-01 — Config gate: surfaces are 404 when disabled • Edge (curl)

Gherkin

```
Given PublicApi:Enabled and Webhooks:Enabled are false (the default)
When I call any /api/public, /api/apikeys or /api/webhooks route
Then it does not exist (404) – the routes aren't mapped and the API-key scheme isn't added
```

Walkthrough

- 1 With both flags **unset/false**, restart the API and call `curl -k https://localhost:7160/api/public/openapi.json` and `.../api/apikeys` (with a JWT).
- 2 **Expected: 404** for both. Now set the two flags true, restart, and re-check — they become live (401/200). Leave them **on** for the rest of this section.

QA-API-02 — Mint an API key (shown once) • Core (curl)

Gherkin

```
Given I am the owner with PUBAPI enabled
When I create an API key
Then I receive the raw pk_... key exactly once, and listing later shows only its prefix/metadata
```

Walkthrough

- 1 `curl -k -X POST https://localhost:7160/api/apikeys -H "Authorization: Bearer <JWT>" -H "Content-Type: application/json" -d '{"name":"qa","scopes":["read"]}'`
- 2 **Expected: 201** with a key field like `pk_...` — **copy it now** (never shown again). `GET /api/apikeys` lists it with `prefix/scopes` but **no** key.
- 3 **Non-owner**: repeat as a member/admin -> **403**.

QA-API-03 — Call the public API with the key; scopes + tenant-scoping • Core (curl / Postman)

Gherkin

```
Given a read-only API key for my tenant
When I call the public API with it
Then whoami returns my tenant, and a write-scoped route is refused (403 insufficient_scope)
```

Walkthrough

- 1 `curl -k https://localhost:7160/api/public/whoami -H "X-API-Key: pk_..." -> 200`, body shows my `tenant_id`, the key name, and `["read"]`.

- 2 `curl -k -X POST https://localhost:7160/api/public/echo -H "X-API-Key: pk_..." -d '{"message": "hi"}'` with the **read-only** key -> ****403 insufficient_scope****. (A key created with "write" succeeds.)
- 3 **No/blank/garbage key -> 401. Revoked key** (DELETE /api/apikeys/{id}) -> **401** afterwards.
- 4 **Postman:** import /api/public/openapi.json, set an X-API-Key header on the collection, run whoami.

QA-API-04 — Per-key rate limit • Core (curl)

Walkthrough

- 1 Fire whoami with one key ~65 times in a minute (for i in \$(seq 1 65); do curl -k -s -o /dev/null -w "%{http_code}\n" https://localhost:7160/api/public/whoami -H "X-API-Key: pk_..."; done).
- 2 **Expected:** the first 60 are **200**, then **429**. A **second** key still returns **200** (budgets are per key, not shared).

QA-API-05 — Register a webhook + send test + verify signature • Core (curl)

Precondition: a receiver URL that echoes requests — e.g. create one at <https://webhook.site> and copy it. **Gherkin**

```
Given HOOKS is enabled and I own the tenant
When I register my receiver and send a test event
Then my endpoint receives a signed POST I can verify with the secret
```

Walkthrough

- 1 POST /api/webhooks (JWT) with {"url": "<webhook.site URL>", "event_types": ["ping"]} -> **201** with a secret (whsec_...) **shown once** — copy it.
- 2 POST /api/webhooks/{id}/test (JWT) -> **200** { "delivered": true, "status_code": 200 }.
- 3 On webhook.site, confirm the request has headers ****X-Webhook-Id, X-Webhook-Event: ping, and X-Webhook-Signature: sha256=...** **Verify: HMAC-SHA256 of the raw body**** with your whsec_... secret equals the signature (any HMAC tool, or the app's WebhookSignature.Compute).
- 4 **Non-owner** management -> **403**. **Bad URL / no event types** on create -> **400**.

QA-API-06 — Delivery log + replay • Core (curl)

Gherkin

```
Given I've sent test deliveries to my subscription
When I view its delivery log and replay one
Then I see per-attempt rows, and replay re-POSTs the same event to my endpoint
```

Walkthrough

- 1 GET /api/webhooks/{id}/deliveries (JWT) -> a list of attempts, newest first, each with success, status_code, error, event_id.
- 2 POST /api/webhooks/deliveries/{deliveryId}/replay (JWT) -> **202**; webhook.site receives the **same** event again (same X-Webhook-Id, so a real receiver can dedup).
- 3 **Failure path (optional):** point the subscription at a URL that returns 500, send a test -> the log shows success:false and the outbox retries with backoff (watch the API logs), dead-lettering after the cap. Replay of an unknown/other-tenant delivery id -> **404**.

15. Traceability matrix (feature -> cases -> API)

| Feature area | Test cases | Key API endpoints |
|---------------------------|---|---|
| OAuth sign-in (Google/MS) | SMK-02, AUTH-02, DSK-02, AND-02 | GET /api/auth/login/{provider}, GET /api/auth/callback/{provider}, native login/callback/exchange |
| Magic link (web) | AUTH-01, 05, 06, MAIL-02 | POST /api/auth/magic-link/send, GET /api/auth/magic-link/verify |
| Email OTP | SMK-01/05/06, AUTH-03/04, DSK-01, AND-01, MAIL-01 | POST /api/auth/otp/send, POST /api/auth/otp/verify |

| Feature area | Test cases | Key API endpoints |
|--|---|--|
| Rate limiting / abuse guard | AUTH-11 | POST /api/auth/otp/send, .../magic-link/send, .../otp/verify (429) |
| Session / refresh / sign-out | SMK-03/04, DSK-03/06, AND-03, SEC-03, SEC-05 (legacy-cookie self-heal) | POST /api/auth/refresh, POST /api/auth/logout, GET /api/auth/me |
| Enumeration / error handling | AUTH-04/07/08/09 | (send + verify endpoints; /login query states) |
| Onboarding (auto tenant) | ONB-01/02, SMK-01 | (provisioned on first auth) |
| Household view/rename | HH-01/02 | GET /api/household, PUT /api/household |
| Members (remove/leave/transfer/dissolve) | HH-03..08 | DELETE /api/household/members/{id}, POST /api/household/leave, POST /api/household/transfer-ownership |
| Invitations | INV-01/06/07/08, MAIL-03 | POST /api/household/invitations, GET /api/household/invitations, POST .../{id}/regenerate, DELETE .../{id} |
| Join / accept | INV-02/03/04/05, DSK-08 / AND-08 (paste invite code — NATIVE-4b; web variant E2E) | POST /api/household/invitations/accept |
| Linked accounts | SET-01..06, DSK-05 | GET /api/auth/logins, POST /api/auth/link/{provider}, DELETE /api/auth/logins/{provider} |
| Localization | I18N-01..04, DSK-09 / AND-09 (native restart persistence — NATIVE-5) | PUT /api/auth/locale (+ resx) |
| Emails / branding | MAIL-01..04, I18N-04 | (SMTP via Mailpit) |
| Tenant isolation / auth guards | SEC-01..05 | (all [Authorize] endpoints; write-stamping + reuse detection are automated) |
| Platform health / readiness | SMK-07 | GET /health, GET /health/ready |
| Transactional email delivery | (all email cases) | async via the outbox dispatcher (OutboxMessages) |
| Billing — checkout/portal/webhook + billing page | BILL-01/02 (\$10c, E2E BillingJourneyTests) + DSK-11 / AND-11 (native refresh-on-return — NATIVE-4) + Api.Tests (Billing*/Entitlement* tests) | POST /api/billing/checkout, .../portal, .../webhook |
| Billing — quotas (BILLING-5) | HH-14 (seat limit blocks invite -> 402 upgrade message) + Api.Tests (QuotaServiceTests) | seats (members + pending invites vs Plan.SeatLimit) enforced on POST /api/household/invitations -> 402 seat_limit_reached; metered usage via IQuotaService.TryConsumeAsync (monthly UsageCounter). Limits in PlanCatalog (null = unlimited). |
| Billing — trial/dunning (BILLING-6) | covered by Api.Tests (BillingWebhookHandlerTests, SubscriptionLapseSweepJobTests); manual via Stripe test triggers | webhook transition into past_due/canceled -> owner notification (in-app bell + outbox email, NOTIFY) once; SubscriptionLapseSweepJob (6h) nudges the owner once when a paid period lapses without a webhook (LapseNotifiedAt). Verify with stripe trigger invoice.payment_failed (test mode) -> owner sees a billing notification in the bell. |
| Billing — dissolve cleanup (BILLING-7) | covered by Api.Tests (BillingDissolveTests) | on tenant dissolve, BillingDataContributor wipes the Subscription projection and enqueues a "billing.cancel" outbox message -> IBillingProvider.CancelSubscriptionAsync (a deleted tenant stops being billed). HasDataAsync=false (billing never blocks leaving); export gains a billing section (plan/status/period, no Stripe ids). Manual (Stripe test mode): subscribe a throwaway tenant, delete the account, confirm the Stripe subscription is canceled. |
| Feature area | Test cases | Key API endpoints |
| Public API + API keys (PUBAPI, config-gated off) | QA-API-01..04 (curl/Postman) + Api.Tests (ApiKeyServiceTests, RateLimitingTests); boot-verified on/off | PublicApi:Enabled toggles it. Owner-only /api/apikeys (create -> raw pk_... once, list, revoke; Permission.ManageApiKeys); API-key auth scheme mints a tenant_id-scoped principal; demo /api/public/whoami (read scope) + /api/public/echo (write scope) via .RequireApiScope. PUBAPI-2 : per-key |

| Feature area | Test cases | Key API endpoints |
|---|---|--|
| | | rate limit (60/min, isolated per key -> 429) + a leak-free public OpenAPI doc at /api/public/openapi.json (only the public routes). Off (default) => routes 404. Manual: PublicApi__Enabled=true, mint a key, curl -H "X-API-Key: pk_..." /api/public/whoami; fetch /api/public/openapi.json. |
| Outbound webhooks (HOOKS, config-gated off) | QA-API-01, 05, 06 (curl/webhook.site) + Api.Tests (WebhookSubscriptionServiceTests, WebhookDeliveryTests, WebhookDeliveryLogTests) + Core.Tests (WebhookSignatureTests); boot-verified on/off | Webhooks:Enabled toggles it. Owner-only /api/webhooks (register -> signing secret whsec_... once, list, delete, send test ; Permission.ManageWebhooks). IWebhookPublisher.PublishAsync fans out to matching active subs -> one "webhook" outbox message each -> signed POST (X-Webhook-Signature), retry/dead-letter via the outbox. HOOKS-2 : a delivery log (GET /api/webhooks/{id}/deliveries — one row per attempt, success/status/error) + replay (POST /api/webhooks/deliveries/{id}/replay — re-enqueue the exact payload). Off (default) => routes 404. Manual: Webhooks__Enabled=true, register a receiver (e.g. a webhook.site URL), hit send test , view deliveries, replay one. |
| Audit log (API-only) | covered by Api.Tests (AuditLogTests) | append-only IAuditLog + interceptor |
| RBAC roles (admin tier) | HH-09/10/11/12 (web roster promote/demote + admin capability/limits); Api.Tests (RolePermissionsTests, PermissionServiceTests, MemberRoleManagementTests) | PUT /api/household/members/{id}/role (owner-only; admin<->member, owner via transfer only); permission seam gates tenant writes |
| File storage (API-only) | covered by Api.Tests (LocalDiskFileStorageTests, FileDownloadTokenizerTests, FilesControllerTests, S3FileStorageMinioTests [real MinIO], FileStorageRegistrationTests) | IFileStorage (tenant-scoped keys; local disk / S3-compatible — AWS/MinIO/R2/B2, config-gated); local signed GET /api/files/{token} (expiring, single-key, tenant-checked -> 404 on any failure); S3 native presigned URLs |
| GDPR data export | HH-13 (owner Household -> Data -> download, E2E GdprExportJourneyTests) + DSK-10 / AND-10 (native share — NATIVE-3) + Api.Tests (TenantExportTests) | POST /api/household/export (owner-only ExportData -> 403 else; JSON bundle via IFileStorage, signed URL; secret-free, tenant-scoped, audited) |
| GDPR account erasure | SET-07 (Settings -> Danger zone) + Api.Tests (AccountErasureTests) | DELETE /api/auth/me (wipes identity/PII in one tx; owner-with-members -> 400, solo owner -> 409 without confirm_dissolve; member removed not re-homed; audited; audit trail survives) |
| MFA / TOTP | MFA-01..05, DSK-12 / AND-12 (native) (Settings enroll/QR/confirm/recovery + disable; step-up on OTP, OAuth/magic-link, and native logins) + Api.Tests (MfaServiceTests, MfaChallengeServiceTests, MfaLoginServiceTests) | GET POST /api/auth/mfa[/enroll /confirm /disable] (enroll/manage; secret encrypted, hashed single-use recovery codes) + login step-up POST /api/auth/mfa/verify (MFA-on logins get a signed challenge instead of a session; verify a TOTP/recovery code to complete). Every sign-in path enforces it (web + native): OTP returns the challenge as JSON; OAuth callback + magic-link redirect to /login?mfa=<challenge>; native OTP/OAuth-exchange return the challenge in the body and the MAUI client steps up in-app. |
| In-app notifications | NOTIF-01..03, DSK-13 (header bell: list/unread-count/mark-read; Settings delivery-preference switches) + Api.Tests (NotificationServiceTests, NotificationFanOutTests) | GET /api/notifications (+?before=&limit=), /unread-count, POST /{id}/read, /read-all, and GET PUT /api/notifications/preferences — per-user (scoped to the caller). NotifyAsync fans out to in-app + email (outbox-backed) per prefs (default both on). |
| Admin back-office | ADMIN-01..03, DSK-14 (native spot) (staff /admin console: tenant list/detail + impersonate w/ banner + stop) + Api.Tests (PlatformStaffServiceTests, AdminControllerTests) | GET /api/admin/me (staff probe, 200 {is_staff} for any caller — drives the nav/gate), GET /api/admin/tenants (+/{id}) inspection, POST /api/admin/impersonate/{userId} — platform-staff only (config Admin:StaffEmails; non-staff -> 403). Detail enters the target tenant (filter never loosened); impersonation returns a short-lived, non-refreshable token with an impersonated_by claim, audited in the target's tenant . |

Per-client coverage: Web = full (all suites). Desktop = DSK-01..14 (auth + per-feature parity). Android = AND-01..13 (auth + per-feature parity incl. hardware back + share sheet). iOS = IOS-01..04 and macCatalyst = MAC-01..03 (first-run

smoke — never run before; needs a Mac). Magic link is **web-only** by design; the §13c checklist is the per-release native subset.

Automated (E2E CI): the following manual cases have an equivalent Playwright/PHPUnit journey in `tests/E2E.Tests`, run on every push by the `e2e` job (`.github/workflows/ci.yml`) against a booted Postgres + Mailpit + API + Web stack — so they are continuously regression-guarded on **Web**:

| Manual case | E2E test |
|---|--|
| QA-SMK-01 (OTP sign-in happy path) | <code>AuthFlowTests.Otp_SignIn_LandsInTheApp</code> (+ <code>Login_Page_Renders</code>) |
| QA-SMK-03 (sign out) | <code>AuthFlowTests.SignOut_ReturnsToLogin</code> |
| QA-AUTH-09 (email-format validation) | <code>AuthFlowTests.Invalid_Email_IsRejected_NoCodeStep</code> |
| QA-MFA-01 (enable TOTP) | <code>MfaJourneyTests.Enroll_ThenStepUp_OnNextSignIn</code> (enroll leg) |
| QA-MFA-02 (step-up enforced at sign-in) | <code>MfaJourneyTests.Enroll_ThenStepUp_OnNextSignIn + StepUp_WithWrongCode_DoesNotSignIn</code> |
| QA-I18N-01 (switch language on login) | <code>I18nTests.Switching_Language_ReRendersTheUi</code> |
| QA-DSK-01 (desktop OTP sign-in) | <code>NativeSmokeTests</code> — the <code>native-smoke-windows</code> job boots the REAL Windows exe and drives it over WebView2 CDP (OTP + household load) |
| QA-AND-01 (Android OTP sign-in) | <code>tests/native-smoke-android/smoke.js</code> — the <code>native-smoke-android</code> job boots a real emulator and drives the app via playwright-core's <code>_android</code> module |

The two native smoke jobs run on develop pushes that touch native-relevant paths (see the `native-paths` gate in `ci.yml`) — they are boot-and-sign-in canaries, not the per-feature native regression, which stays manual (§12–13b). All other cases remain manual-only or API-test-backed as noted per row.

16. Sign-off sheet

Record one row per executed case. Build = API/web commit SHA (`git rev-parse --short HEAD`).

| Case ID | Client | Result (P/F/Blocked/N-A) | Tester | Build (SHA) | Date | Notes / defect link |
|-----------|--------|--------------------------|--------|-------------|------|---------------------|
| QA-SMK-01 | Web | | | | | |
| QA-SMK-02 | Web | | | | | |
| ... | | | | | | |

Release gate (suggested): all • **Smoke** Smoke + all • **Core** Core cases Pass on Web; the §13c native checklist Pass on every platform being shipped (iOS/macCatalyst once those ship); no open Critical/High defects. • **Edge** Edge cases triaged (Pass or accepted-known-issue).

17. Notes for maintainers

- This plan is the manual counterpart to the automated `tests/E2E.Tests` (Playwright/PHPUnit) suite. That suite now covers the **OTP auth happy-path + guards** (`AuthFlowTests`), the **MFA enroll -> step-up** journey incl. the wrong-code negative (`MfaJourneyTests`), and the **language switch** (`I18nTests`) — all over the page objects in `Pages/`, reading codes from Mailpit via the `Mailpit` REST client, and **run in CI** by the `e2e` job (`.github/workflows/ci.yml`, Web only). See `tests/E2E.Tests/README.md` for the run procedure (it documents the same Mailpit SMTP override noted in §1.1). The cases these journeys mirror are marked **Automated in CI** and listed in §15. The Gherkin blocks here are written to be lifted directly into new E2E scenarios — keep the two in sync as automation grows.
- When you add an app-specific domain feature on top of this platform, add a matching suite here and a row in the traceability matrix (§15) so "entire functionality" stays honest.
- `docs/FEATURES.md` describes the same JWT-based flows at the design level; this plan is their step-by-step verification. Keep the two in sync when behavior changes.
- Updated 2026-06-22** for the security/quality remediation: OTP lockout is now **cumulative per email** and resend-proof (QA-AUTH-04); passwordless send/verify are **rate-limited** (QA-AUTH-11); the Settings **Unlink** now requires a fail-closed confirmation (QA-SET-04); and server-side hardening (write-stamping, refresh-reuse detection, fail-closed takeover guard) is captured as automated coverage (QA-SEC-05).

- **Updated 2026-06-23:** QA-AUTH-02 now documents the **tenant-gated same-email auto-link** rule (Microsoft trusted only on the consumers tenant; work/school + unknown providers fail closed — audit MITI-3); and QA-SEC-05 adds the **legacy refresh-cookie self-heal** as automated coverage (`CookieServiceTests`).
- **Updated 2026-06-26** for the platform-foundation work (JOBS/BILLING/OBS, ADR-006/007/008):
- **Transactional email is now delivered asynchronously** via the outbox dispatcher — a few-second delay, and the send request always succeeds (reliability moved to the background). Affects every email-based case; see the §1.1 and §11 notes. The `E2E AuthFlowTests` (which reads OTP codes from Mailpit) now traverses this async path — confirm it waits/polls for the email rather than assuming instant delivery.
- Added **API health/readiness** endpoints + smoke case **QA-SMK-07** (`/health`, `/health/ready`).
- The **billing API** (checkout/portal/webhook), the **append-only audit log**, and **OpenTelemetry** telemetry are API/operational-level with **no client UI** — covered by `tests/Api.Tests`, E2E pending; manual cases will follow when UI exists (see §2 + §15).
- **Updated 2026-06-30** for RBAC (ADR-009): the tenant role model gains an ****admin tier behind a permission seam (owner > admin > member)**. **RBAC-1 added the seam (no behavior change)**; **RBAC-2 added the owner-only role-change endpoint**** (`PUT /api/household/members/{id}/role`, `admin<->member`; owner is conferred only via transfer) and hardened member-removal so the **owner can't be removed**. This was API-only at first; **RBAC-3 added the web UI** — the Household roster now has owner-only **Make admin / Make member** controls, and the page is admin-aware (admin can rename + invite/remove members, but sees no role/transfer/dissolve controls). New web cases **QA-HH-09..12**; §7 intro updated for the three-tier model. API coverage stays automated (`tests/Api.Tests`).
- **Updated 2026-06-30** for file storage (ADR-010): an `IFileStorage` seam (tenant-scoped keys, local-disk dev default / S3-compatible prod) with a signed, time-limited download endpoint `GET /api/files/{token}`. **API-only** (no UI consumer yet) — covered by `tests/Api.Tests` (`LocalDiskFileStorageTests`, `FileDownloadTokenizerTests`, `FilesControllerTests`); see §2 + §15. Manual cases will follow when a feature (avatars/attachments) wires it to UI. FILES-3 added the **S3-compatible** backend (AWS/MinIO/R2/B2, config-gated by `Storage:S3:Bucket`), tested against a real **MinIO** container (`S3FileStorageMinioTests`); prod config is documented in `.env.example`. **This completes Wave 1** (RBAC + File storage).
- **Updated 2026-07-01** for GDPR data export (ADR-011, Wave 2): owner-only `POST /api/household/export` assembles the tenant's data (core + each feature's contributor section) into a JSON bundle stored via file storage and returns a **signed download URL**; **secret-free** (no token hashes), tenant-scoped, audited. **API-only** (no UI button yet) — covered by `Api.Tests` (`TenantExportTests`); see §2 + §15.
- **Updated 2026-07-01** for GDPR account erasure (ADR-011): `DELETE /api/auth/me` ("delete my account") wipes the caller's identity/PII (`User/UserLogin/RefreshToken/LoginToken`) in one transaction, single-owner-safe (owner-with-members -> transfer first; solo owner -> confirm to dissolve; member removed, not re-homed), audited (the audit trail survives — actor ids, never PII). **API-only** — covered by `Api.Tests` (`AccountErasureTests`). **GDPR epic complete (export + erasure)**.
- **Updated 2026-07-01** for MFA enrollment (ADR-012, Wave 2): authenticator-app **TOTP** (`Otp.NET`) — `/api/auth/mfa/*` (enroll -> provisioning URI/QR; confirm with a code -> enable + one-time recovery codes; disable; status). Secret **encrypted at rest** (Data Protection); recovery codes **hashed + single-use**; MFA rows wiped by account erasure. **API-only** (MFA-1) — covered by `Api.Tests` (`MfaServiceTests`). **MFA-2 (login step-up)**: an MFA-on login returns a **signed challenge** instead of a session; `POST /api/auth/mfa/verify` completes it with a TOTP/recovery code. Wired into **OTP-verify + native-exchange**; the OAuth/magic-link **redirect** paths route to a client `/mfa` page (needs the MFA UI) and are a flagged **follow-up** — no platform-web user can enable MFA via those paths today (no enrollment UI). Covered by `MfaChallengeServiceTests` + `MfaLoginServiceTests`. **This completes Wave 2** (GDPR + MFA).
- **Updated 2026-07-01** for in-app notifications (ADR-013, Wave 3): a **per-user** notification center — `GET /api/notifications` (paginated), `/unread-count`, `POST /{id}/read`, `/read-all` — scoped to the caller (never cross-user); features produce via `NotifyAsync` (staged in-app row). Notifications are user PII (wiped by account erasure). **API-only** — covered by `Api.Tests` (`NotificationServiceTests`). **NOTIFY-2: per-user delivery preferences** (`GET|PUT /api/notifications/preferences`, default both on) + `NotifyAsync` **fan-out** — in-app row + email via the outbox-backed `IEmailSender`, gated by prefs. Covered by `NotificationFanOutTests`. Bell-menu UI is an API-first follow-up. **NOTIFY epic + Wave 3's first item complete**.
- **Updated 2026-07-01** for the platform-staff admin surface (ADR-014, Wave 3): `GET /api/admin/tenants (+ /{id})` — **staff-only** (config allowlist `Admin:StaffEmails`, out-of-band; non-staff -> 403). Read-only cross-tenant inspection; the per-tenant detail **enters the target tenant** (the global filter is never loosened) and is **audited in that tenant**. **API-only** — covered by `Api.Tests` (`PlatformStaffServiceTests`, `AdminControllerTests`).

- **Updated 2026-07-01** for admin **impersonation** (ADR-014, ADMIN-2): POST `/api/admin/impersonate/{userId}` (staff-only) returns a **short-lived (15-min), non-refreshable** access token carrying the target's identity + an `impersonated_by` claim; **loudly audited in the target's tenant**. Unknown target -> 404. Covered by `AdminControllerTests`. **This completes the ADMIN epic — and the planned platform** (nine epics, ADRs 006–014).
- **Updated 2026-07-01** — UI pass (putting a face on the API-first surfaces). **UI-1 (GDPR):** the owner **Data -> Download household data** button on Household (**QA-HH-13**) and **Settings -> Danger zone -> Delete my account (QA-SET-07)** — wired to POST `/api/household/export` and DELETE `/api/auth/me` (single-owner-safe, with the dissolve second-confirm). EN/ES localized.
- **Updated 2026-07-01** — **UI-2 (MFA):** a **Two-factor authentication** card in Settings (MfaCard component) — enroll (POST `/api/auth/mfa/enroll`) renders a **client-side QR** of the `otpauth://` URI (vendored `qrcode-generator`, MIT, in `Shared.Ui/wwwroot/js/`; the secret never leaves the browser) alongside a manual key, confirm (`/confirm`) reveals one-time **recovery codes**, and **disable** (`/disable`) needs a live code. The **Login** page now handles the OTP-verify `mfa_required` response with a **step-up code prompt** -> POST `/api/auth/mfa/verify` (TOTP or recovery code) -> `/auth-callback`. **QA-MFA-01..03**; EN/ES localized. Native OTP + OAuth/magic-link step-up remain follow-ups (web-first).
- **Updated 2026-07-01** — **UI-3 (Notifications):** a **header bell** (NotificationBell component) — unread-count badge (polled ~60s), a dropdown list (newest-first, unread dot + relative time), click-to-mark-read and **Mark all read** via GET `/api/notifications[/unread-count]`, POST `/api/notifications/{id}/read`, `/read-all`. A **Notifications** card in Settings (NotificationPrefsCard) with **In-app/Email** switches (optimistic save, reverts on error) via GET | PUT `/api/notifications/preferences`. **QA-NOTIF-01..03**; EN/ES localized. (No built-in producer — items appear once a feature calls `NotifyAsync`.)
- **Updated 2026-07-01** — **UI-4 (Admin console):** a staff-only `/admin` page (AdminConsole) — tenant list + detail (members/subscription/audit count) and **Sign in as** (impersonation). Staff detection uses a new **non-gating** probe GET `/api/admin/me` (`{is_staff}` for any authenticated caller — the only API addition in the UI pass; the allowlist stays config-only, actions still 403 for non-staff), so the header shows an **Admin** link only to staff. Impersonation swaps the in-memory session to the short-lived token (`AuthService.BeginImpersonation`), pins an **impersonation banner** in `MainLayout` (reads the `impersonated_by` claim), and **Stop impersonating** restores the staff identity from the refresh cookie; a reload also reverts (the token is non-refreshable). **QA-ADMIN-01..03**; EN/ES localized. **This completes the UI pass (UI-1..4).**
- **Updated 2026-07-01** — **MFA-3 (redirect step-up, security fix):** OAuth callback and magic-link verify now route through `IMfaLoginService.CompleteOrChallengeAsync` (like the OTP path) instead of issuing a session directly — closing the gap where an MFA-enabled user could sign in via Google/ magic link and **skip the second factor**. When MFA is on, the server redirects to `/login?mfa=<challenge>` (a signed, single-use, 5-min Data-Protection token — no secret), and `Login.razor` reuses the UI-2 step-up prompt -> POST `/api/auth/mfa/verify` -> `/auth-callback`. Property covered by `MfaLoginServiceTests` (MFA-on -> challenge, never a session); **QA-MFA-04**. The dead `IssueRefreshCookieAsync` helper was removed.
- **Updated 2026-07-01** — **MFA-4 (native step-up):** the MAUI client now handles the `{mfa_required, challenge}` response on the native **OTP** and **OAuth-exchange** paths. `AuthService.VerifyOtpAsync/SignInWithOAuthAsync` now return a `SignInResult` (Success / Failed / MfaRequired+challenge) and a new `VerifyMfaAsync` completes the step-up (tokens in the body, native transport); `Login.razor`'s native branches reuse the same code prompt. Client-only — no API change; build-verified (native is E2E/manual per `MOBILE_TESTING.md`); **QA-MFA-05**. **MFA is now enforced on every sign-in path, web and native — the epic is fully closed, no open gaps.**
- **Updated 2026-07-01** — **BILLING-5 (quotas):** `IQuotaService` adds plan **seat** limits (members + pending invites vs `Plan.SeatLimit`, enforced on the invite path -> ****402 seat_limit_reached, with an upgrade message in the Household invite UI**) and metered usage** (`TryConsumeAsync` against a monthly, self-resetting `UsageCounter`). Limits are `PlanCatalog` data — null/absent = unlimited, so it's inert until set (platform ships example caps: Free 3/3, Pro 10/100). New entity + migration `AddUsageCounter`. Covered by `QuotaServiceTests` (10 cases); **QA-HH-14**; EN/ES. Only **BILLING-6** (trial/dunning) and a billing-dissolve contributor remain from the BILLING epic.
- **Updated 2026-07-01** — **BILLING-6 (trial/dunning):** the owner-facing reaction to the subscription lifecycle. `IBillingNotifier` notifies the tenant **owner** via the notification center (in-app bell + outbox email). The **webhook** notifies once on a transition into `past_due/canceled` (no spam; inbox dedups). A ****SubscriptionLapseSweepJob**** (6h) nudges once when a paid period lapses without a webhook, recording `Subscription.LapseNotifiedAt` (migration `AddSubscriptionLapseNotifiedAt`) — Stripe stays source of truth, no fabricated status. Covered by `BillingWebhookHandlerTests` +

SubscriptionLapseSweepJobTests; manual via stripe trigger. **The BILLING epic is now complete** (1–6); only optional follow-ups remain (advance trial nudge; billing-dissolve contributor).

- **Updated 2026-07-01 — PUBAPI-1 (public API + API keys, config-gated OFF):** a programmatic surface for machines (the user reversed the earlier "no public API" stance). `ApiKey` (hash-only, `pk_...` revealed once; migration `AddApiKey`); a second **API-key auth scheme** mints a `tenant_id`-scoped principal (so tenant isolation applies for free); owner-only `/api/apikeys` management (`Permission.ManageApiKeys`); demo `/api/public/whoami` (read) + `/echo` (write) gated by `.RequireApiScope`. ****PublicApi:Enabled** (default false) — strong gating: off => the scheme isn't added and the routes 404. ****Covered by `ApiKeyServiceTests` (9); boot-verified on (401 without a key) and off (404).** **HOOKS** (outbound webhooks) is the companion outbound half — next.
- **Updated 2026-07-01 — HOOKS-1 (outbound webhooks, config-gated OFF):** the outbound integration half. `WebhookSubscription` (url + event types + **encrypted** signing secret, revealed once; migration `AddWebhookSubscription`); `IWebhookPublisher.PublishAsync` fans out to matching active subs -> one "webhook" **outbox** message each (durable, retried, atomic with the change); `WebhookOutboxHandler` signs (HMAC-SHA256, X-Webhook-Signature) + POSTs, throwing on non-2xx so the outbox retries/dead-letters. Owner-only `/api/webhooks` (`Permission.ManageWebhooks`) with a synchronous **send test**. ****Webhooks:Enabled** (default false) — off => routes 404. ****Covered by `WebhookSignatureTests` (3) + `WebhookSubscriptionServiceTests` + `WebhookDeliveryTests` (11 in `Webhooks/`); boot-verified on (401) and off (404).** **This completes the integration story (PUBAPI inbound + HOOKS outbound), both default-off.**
- **Updated 2026-07-01 — PUBAPI-2 (hardening): per-key rate limiting** (`RateLimiting.PublicApiPolicy` partitions on the key id — 60/min, one key can't exhaust another's budget -> 429; `RateLimitingTests` proves per-key isolation) and a **leak-free public OpenAPI doc** (GET `/api/public/openapi.json`, anonymous, emits only the `/api/public` routes so the internal v1 surface is never exposed; boot-verified it serves in Production when enabled and 404s when off).
- **Updated 2026-07-01 — HOOKS-2 (delivery log + replay):** a tenant-facing debug trail. `WebhookDelivery` records **one row per delivery attempt** (event, success, status/error, and the sent body; not `ITenantScoped` — written from the tenant-less dispatcher, read side filters by `TenantId`; migration `AddWebhookDelivery`). Owner routes GET `/api/webhooks/{id}/deliveries` + POST `/api/webhooks/deliveries/{id}/replay` (re-enqueue the exact stored payload, same event id). Covered by `WebhookDeliveryLogTests`. **Candidate HOOKS-3 (not built):** a Blazor management UI for webhooks/API keys.
- **Updated 2026-07-01 — BILLING-7 (dissolve cleanup):** closed the one real gap — deleting a billing-enabled tenant left the Stripe subscription active (still charging). `BillingDataContributor` now wipes the `Subscription` projection on dissolve and **Cancels the provider subscription** via a "billing.cancel" outbox message (`BillingCancelOutboxHandler` -> new idempotent `IBillingProvider.CancelSubscriptionAsync`) — out-of-band, not an external call inside the teardown tx. `HasDataAsync=false` so billing never blocks leaving; export gains a secret-free billing section. Covered by `BillingDissolveTests` (6). **Closes the BILLING epic (1–7).**
- **Updated 2026-07-01 — added §14b — manual QA for the API surfaces (PUBAPI + HOOKS):** curl/Postman cases (QA-API-01..06) for the config gate, minting/using API keys (scopes, tenant-scoping, rate limit), and registering webhooks (send-test, signature verification, delivery log, replay). These surfaces have **no web UI by design** (they're for machines), so this is the human-testable complement to the automated tests — the Postman path is one import of `/api/public/openapi.json` away. Referenced from the §15 PUBAPI/HOOKS rows and the §2 "no client UI" note.
- **Updated 2026-07-02 — E2E-in-CI (v2 audit B8-5/B8-6):** the Playwright suite is now booted and run on every push (the e2e job in `.github/workflows/ci.yml` — Postgres + Mailpit + API + Web) and was expanded beyond the auth happy-path to the **MFA enroll -> step-up** journey (`MfaJourneyTests`, incl. a wrong-code negative) and the **login-page language switch** (`I18nTests`). Six manual cases now have a continuously-run Web equivalent and are marked **Automated in CI**: QA-SMK-01, QA-SMK-03, QA-AUTH-09, QA-MFA-01, QA-MFA-02, QA-I18N-01 (mapping table added under §15; how-to note in §2/§1 "How to use"). These stay in the manual plan for **Desktop/Android** (CI runs Web only) and area-change re-runs; human QA can spot-check them on Web rather than run them in full each cycle. `data-testid` hooks were added to `MfaCard` and `LanguageSwitcher` to keep the selectors stable.

- **Updated 2026-07-03** for NATIVE Wave 2 (ADR-018, docs/NATIVE_PARITY.md): the parity audit's six gaps were fixed (join-by-code, culture persistence, export download/share, billing return-refresh, Android hardware back — plus **G7**, iOS/macCatalyst crashing at boot, found while writing this expansion) and this plan grew the per-feature native coverage that verifies them on hardware: **QA-DSK-08..14**, **QA-AND-07..13**, the first-ever **iOS/macCatalyst smoke** (§13b, QA-IOS-01..04 + QA-MAC-01..03), and the **per-release native checklist** (§13c). The web halves of the fixes are already -automated (Member_Joins_By_Pasting_The_Invite_Code, Owner_Downloads_The_Data_Export); the OS-chrome halves (share sheets, hardware back, real focus-return, safe areas) are exactly what these manual cases exist for.
- **Updated 2026-07-06 — first Apple run of §13b** (maintainer's MacBook Air M1, iOS 26.5 simulator + Mac Catalyst): **QA-IOS-01 PASS** (boots to login — validates the G7 fix on a real Apple runtime), **QA-IOS-02 PASS**, **QA-IOS-04 PASS** (first-ever exercise of the ASWebAuthenticationSession -> perezosoftware://auth path; also verified on iPhone 17 Pro Max + iPad Air 11" simulators), **QA-MAC-01 PASS**, **QA-MAC-02 PASS** and the OAuth leg of QA-MAC-03 PASS. Remaining: the core-flows spot-checks (QA-IOS-03, rest of QA-MAC-03 — share sheet, language + restart persistence). The pass surfaced two platform gaps, fixed in the same PR as this entry: (1) macOS trust evaluation fails Brevo's SMTP TLS handshake ("incomplete certificate revocation check") -> new Email:Smtp:CheckCertificateRevocation setting, default **on**, dev-box opt-out documented in .env.example (SmtpSettingsTests); (2) sign-in on ad-hoc-signed Mac Catalyst Debug builds died storing the refresh token — MAUI SecureStorage needs the restricted keychain-access-groups entitlement (MissingEntitlement without it, SIGKILL at launch with it) -> store entitlement added to Entitlements.plist for signed builds, Debug builds swap to Entitlements.Debug.plist (unsandboxed) + a DebugFileSessionStore fallback (MACCATALYST && DEBUG only); NATIVE-9 must re-verify SecureStorage under real signing.